LLM ENGINEERING

Week 03

Multi-Head Attention & Positional Encoding

Thursday, 29 October 2026

Interactive version: https://llm.strueber.dev/#/week-03

Bethgelab · University of Tübingen · Winter term 2026/27

Text and figures under CC BY 4.0, code under the MIT licence.

Recap & Roadmap

1

Announcements

Next week, 5 November

- Lecture in person again
- Read Bonus 2 before the tutorial

Quiz 1 on ILIAS

- Weeks 1, 2 and Bonus 1
- Due Friday 30 October, end of day

Second exercise: this lecture

- Due Wednesday 4 November, 22:00
- You get an email when it arrives

▼ Dates for the coming week

Next week's lecture, on 5 November, takes place in person again.

Quiz 1 is on ILIAS and closes at the end of Friday, 30 October. It covers Week 1, Week 2 and Bonus 1.

The second coding exercise covers the content of this lecture and is due on Wednesday, 4 November, at 22:00. You will get an email when it is in your pair's repository.

Bonus 2, the chapter on long contexts, is required reading. The Week 4 tutorial discusses its reflection questions, so read it before then.

Week 2 built one head and one block

- One attention head, width `D`
- One pre-norm block, final RMSNorm, tied head
- Position added once, at the input
- Each query: one softmax row

▼ **Week 2's model gives each query one softmax row**

Last week you built a complete language model around a single attention head. Write B for the batch size, S for the number of prediction positions, D for the model width and V for the vocabulary size. A batch began as raw token windows of shape `[B,S+1]`. One shift turned each window into inputs and next-token targets, both of shape `[B,S]`. A token lookup gave each input token a vector of width D, and a sinusoidal position signal was added to that vector. Position therefore entered the model once, at the input. The result was the hidden stream X of shape `[B,S,D]`. One pre-norm block transformed the stream without changing its shape. A final RMSNorm and the output head, which is tied to the embedding table, then produced logits of shape `[B,S,V]`.

The block held one attention head of width D. Week 2 derived how that head works, and this lecture builds on the derivation. You also estimated what the model costs in parameters, FLOPs and memory, and checked each estimate against a measurement. This week keeps that habit.

The single head is where this week starts. Take the query at position m. The head computes one row of softmax weights a_{mn} over the keys at positions $n \leq m$. The weights are nonnegative and sum to one, $\sum_{n \leq m} a_{mn} = 1$, and the head's output at position m is the weighted average $\sum_{n \leq m} a_{mn} v_n$ of the value vectors. Everything the head gathers for this query passes through that one row of weights.

This week extends three Week 2 choices: the number of heads, the number of blocks and the way position enters.

How does attention read several things at once, and know where they are?

- Heads: several reads per query
- Shared key and value heads: smaller cache
- One mask for padded, packed batches
- Position inside the score: rotary embeddings
- Exercise: six variants, one comparison

▼ Today's question

The question for this lecture is: how does attention read several things at once, and know where they are? The Week 2 head gives each query one weighted average. It learns about position only from a signal added at the input. This lecture changes both, in six parts.

1. Attention gets several heads, so that each query gets several reads at the width of one.
2. Query heads share key and value heads. A model that generates text stores the keys and values of every earlier token, and sharing heads makes that cache smaller.
3. Real batches contain padded sequences and several documents packed into one row. One composed mask makes attention correct for both.
4. The Week 2 block is repeated into a stack of layers.
5. Rotary position embeddings (RoPE) put position into the attention score itself, instead of adding it once at the input.
6. The last part asks what a causal model can learn about position when it receives no explicit position signal.

Heads, shared key and value heads, the mask and RoPE are treated in depth. The stack and the model without position are short.

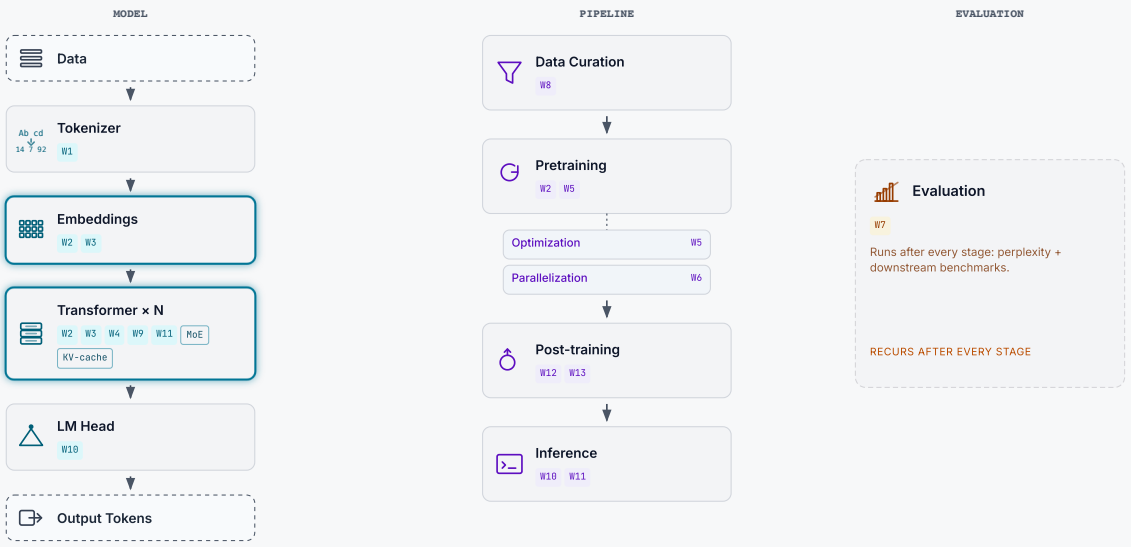
The exercise builds all six parts and trains six variants of the same model:

- the base model, with four heads and RoPE
- a model with a single head
- two models whose four query heads share two key and value heads, or only one
- two models with sinusoidal position, or with none

Every variant trains for the same steps on the same data in the same order, so the variants differ only in their shape. You then compare what each variant costs against what it learns.

Where this fits Read on your own

- Today: attention, position, the stack
- Week 4: which keys, how fast



10 CONNECTIONS

◀ Prev Reset Start ▶

Click "Start" to walk through how decisions in one part of the pipeline shape every other part.

▼ Reading the Week 3 pipeline map

The map places this week in the course. Its highlighted stages are the two model stages that change today: the embeddings and the Transformer × N.

Inside the Transformer, two changes reorganize the attention channels. Multi-head attention runs several heads side by side, and grouped-query attention lets query heads share key and value heads. The composed mask changes which query-key pairs are allowed. The repeated block turns the `× N` label into an actual stack.

Position handling touches both stages. Sinusoidal position belongs to the embeddings, rotary embeddings act inside attention, and a model without position uses neither.

The connection stepper selects one relationship at a time. It highlights the two endpoints of that relationship together with the path between them. The map shows how the course is organized and which parts depend on which. It is a navigation aid and contains no measurements of speed, quality or cause.

Today's changes are attention, position and the stack. Week 4 stays inside attention and asks which keys a query may read and how fast attention runs.

Multi-Head Attention

5

How can attention read two places at once without blending them?

Answer A1

The Idea: Attend in Parallel Subspaces

```
Q, K, V     [B,S,D]
reshape     [B,S,H,D_h]
transpose   [B,H,S,D_h]
scores      [B,H,S,S]
context     [B,H,S,D_h]
transpose   [B,S,H,D_h]
concat      [B,S,D]  → W_O
```

- **Multi-head attention** (MHA): `D_h = D/H` (Vaswani et al., 2017)
- Heads split learned projections
- Scale by `√D_h`, not `√D`

▼ **The shape path and the scale of each head**

Let X, of shape `[B,S,D]`, be the input to attention, with batch size B, sequence length S and model width D. Let H be the number of heads and D_h the number of channels per head, which the exercise code calls `head_dim`. The course sets $D = HD_h$, so $D_h = D/H$. That equality is a choice of the course configuration, and other attention designs need not make it.

The course uses three bias-free projections $W_Q, W_K, W_V \in \mathbb{R}^{D\times D}$, each mapping X to a tensor of shape `[B,S,D]`. Multi-head attention (MHA) splits the output of each projection into H heads and runs attention in every head in parallel (Vaswani et al., 2017). Head h, for $h = 0, \ldots, H-1$, takes the channels hD_h to $(h+1)D_h - 1$. Its queries are $Q_h = XW_Q^{(h)}$, where $W_Q^{(h)} \in \mathbb{R}^{D\times D_h}$ is the matching block of columns of W_Q, and its keys K_h and values V_h are formed in the same way. For one sequence, one MHA layer computes

$$C_h = \operatorname{softmax}\left(\frac{Q_h K_h^\top}{\sqrt{D_h}} + M\right) V_h, \quad h = 0, \ldots, H-1,$$
$$O = \begin{bmatrix} C_0 & C_1 & \cdots & C_{H-1} \end{bmatrix} W_O,$$

where M is the causal mask of Week 2, the softmax runs across the keys of each row, the brackets concatenate along the channels, and $W_O \in \mathbb{R}^{D\times D}$. Week 2 called the blended values H. Now that H counts heads, the blended values of head h are its **context** C_h, and the concatenation is the full context C. Week 2's d_k is D_h with $H = 1$: its single head was as wide as the model, $d_k = D$.

An implementation may fuse the three projections into one matrix $W_{QKV} \in \mathbb{R}^{D\times 3D}$, whose output is split into Q, K and V. It holds the same $3D^2$ parameters as three square matrices and computes the same result.

The shape path

The slide's path computes the same definition, batched over the B sequences and the H heads. The reshape to `[B,S,H,D_h]` splits the D projected channels of each token into H groups of D_h. Each head receives a slice of the learned projections, so what each slice holds is learned too. The split is made in the projected channels and leaves the input X whole: every head sees all of X through its own weights. The transpose to `[B,H,S,D_h]` moves the head axis in front of the sequence axis. Each head is then an independent attention computation over S positions, batched over B and H. The query-key products give the scores `[B,H,S,S]`, one $S \times S$ matrix per head, and multiplying the softmax weights by V gives the context `[B,H,S,D_h]`.

The way back takes two steps. A transpose first returns the context to `[B,S,H,D_h]`, and a reshape then concatenates the heads of each position into `[B,S,D]`. The transpose back is the step a reshape-only version gets wrong. Reshaping `[B,H,S,D_h]` straight to `[B,S,D]` produces the right shape with the wrong content: each output row then holds several positions of one head, instead of all heads of one position. Finally W_O maps the concatenated heads to the block output `[B,S,D]`.

A synthetic example checks the shapes, with $B = 2$, $S = 4$, $D = 8$ and $H = 2$, so $D_h = 4$:

Step	Shape
input X	`[2,4,8]`
Q, K and V, each	`[2,4,8]`
reshape	`[2,4,2,4]`
transpose	`[2,2,4,4]`
scores	`[2,2,4,4]`
context	`[2,2,4,4]`
transpose back	`[2,4,2,4]`
concatenate	`[2,4,8]`
output of W_O	`[2,4,8]`

The scores and the context have the same shape here only because the example has $S = D_h = 4$. Their last axes still mean different things: keys for the scores, channels for the context.

The scale follows the width of a head

Each head divides its scores by $\sqrt{D_h}$, because its dot product runs over D_h coordinates. Week 2 derived the scale for one head of width d_k, and the same argument applies to each head separately.

- **A1. Week 2's variance-preserving assumptions.** At initialization, the inputs have an RMS of about 1, and the entries of W_Q and W_K are independent, with mean 0 and variance $1/D$.

These are assumptions A1 to A3 of Week 2's derivation. A query of head h consists of coordinates of the full query XW_Q, each still a sum over all D input coordinates, so Week 2's step 1 applies unchanged: every coordinate of a head's query and key has mean 0 and variance about 1.

- **A2. The course split.** The heads together have $HD_h = D$ channels, as the course configuration sets.

Two facts are used. The first is Week 2's result for a query and a key of d_k coordinates, under Week 2's A1 to A3,

$$\mathrm{Var}(q \cdot k) \approx d_k, \tag{F1}$$

and the second is how a variance scales under a constant factor c:

$$\mathrm{Var}(cY) = c^2 \,\mathrm{Var}(Y). \tag{F2}$$

Claim. Under A1 and A2, at initialization, the unscaled score $q \cdot k$ of one query and one key of a head has a variance of about D_h. Dividing it by $\sqrt{D_h}$ brings the variance to about 1. Dividing by $\sqrt{D}$ instead makes every score $\sqrt{H}$ times smaller, with a variance of about $1/H$.

Here q and k each have the D_h coordinates of one head:

$$\begin{aligned}
\mathrm{Var}(q \cdot k) &\approx D_h && \text{F1 with } d_k = D_h, \text{ by A1} \\
\mathrm{Var}\left(\frac{q \cdot k}{\sqrt{D_h}}\right) &\approx 1 && \text{F2 with } c = 1/\sqrt{D_h} \\
\frac{q \cdot k}{\sqrt{D}} &= \frac{1}{\sqrt{H}} \cdot \frac{q \cdot k}{\sqrt{D_h}} && \text{A2: } \sqrt{D} = \sqrt{H}\,\sqrt{D_h} \\
\mathrm{Var}\left(\frac{q \cdot k}{\sqrt{D}}\right) &\approx \frac{1}{H} && \text{F2 with } c = 1/\sqrt{H}.
\end{aligned}$$

The second line is the order-one scale that Week 2's division restores. The third line is exact. It holds for every score of every head and uses nothing but A2. The course's actual initialization gives each coordinate a variance of about $1/3$ instead of 1, so Week 2 found $\mathrm{Var}(q \cdot k) \approx D_h/9$. That scales the first, second and fourth lines by $1/9$, but not the exact third line. In Week 2 the single head had $d_k = D$, and the two divisors coincided. With $H > 1$ they differ. At the course configuration, $D = 512$ and $H = 4$, the divisor is $\sqrt{128} \approx 11.3$, not Week 2's $\sqrt{512} \approx 22.6$. Keeping $\sqrt{D}$ is therefore a bug: it divides every score by an extra factor of $\sqrt{H}$, which is 2 here.

7

Which has more parameters?

One head of width 512, or four of width 128?

Answer A2

Each head has QK and OV circuits Read on your own

- QK: which keys get attention (Elhage et al., 2021)
- OV: what gets written back
- Heads can mix several behaviors
- Layers compose: induction heads (Olsson et al., 2022)

▼ Two circuits as a way to analyze a head

Elhage et al. analyze an attention head as two circuits (Elhage et al., 2021). The **QK circuit**, formed by the query and key projections, sets the attention scores. It decides which query-key pairs score high, and so which keys a query attends to. The **OV circuit**, formed by the value and output projections, decides how the attended information is transformed before it is written back into the residual stream.

The circuits as matrices

Both circuits can be read off the definition of MHA. Split W_O by rows into H blocks $W_O^{(h)} \in \mathbb{R}^{D_h \times D}$, one per head, so that block h multiplies the channels of the context C_h in the concatenation. Block multiplication turns the concatenation into a sum,

$$O = \begin{bmatrix} C_0 & \cdots & C_{H-1} \end{bmatrix} W_O = \sum_{h=0}^{H-1} C_h \, W_O^{(h)},$$

so each head writes its own term into the output. Let x_m be the input row at position m. Before the mask and the softmax, the score of head h between query position m and key position n is

$$\frac{x_m \, W_Q^{(h)} W_K^{(h)\top} \, x_n^\top}{\sqrt{D_h}}.$$

Let A_h be the attention weights of head h, of shape `[S,S]` for one sequence. The term the head writes is

$$C_h \, W_O^{(h)} = A_h \, X \, W_V^{(h)} W_O^{(h)}.$$

Without a position rotation, as in this definition, the QK circuit is the $D \times D$ matrix $W_Q^{(h)} W_K^{(h)\top}$, and the OV circuit is the $D \times D$ matrix $W_V^{(h)} W_O^{(h)}$. The QK circuit enters only the weights A_h, so it chooses where the head reads. The OV circuit acts on whatever those weights select, so it

sets what the read contributes. The two are coupled parts of one computation, since the weights from the QK circuit multiply the output of the OV circuit.

The circuits are a way to analyze a head. They describe its computation, which need not amount to one job a person can name. A head can combine several behaviors, the output projection W_O mixes the contributions of all heads, and later layers can build on what earlier heads wrote.

Induction heads compose two layers

Depth lets heads in different layers compose. One studied example is the induction pattern: when a sequence contains `[A][B]` and later `[A]` again, the model predicts `[B]`. A previous-token head composed with an induction head across two layers produces this pattern (Olsson et al., 2022). In the earlier layer, the previous-token head writes into each position which token came before it. In the later layer, the induction head at the second `[A]` compares its query with those writes and attends to the position whose previous token was `[A]`, which is the position of `[B]`. Its OV circuit then copies `[B]` toward the prediction.

Induction is one studied example of composition across layers. It is not a claim about every head, and the exercise does not ask you to find or build an induction head. We use only the two circuits and this composition here. Elhage et al. develop the full mathematical framework.

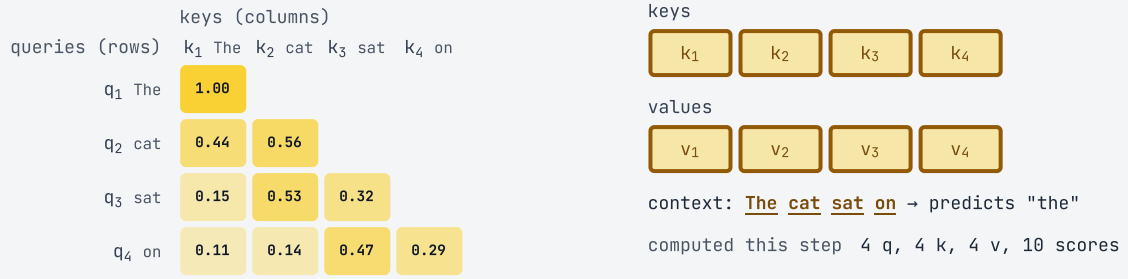
Grouped-Query Attention

9

One more token: what is new?

Which scores, keys and values must be computed?

ILLUSTRATIVE | Prefill | Decode 1 | Decode 2



computed this step

Rows are queries and columns are keys, on the toy sentence from Week 2's heatmap. A filled cell is computed in the current step, and its number and shade give its attention weight. An empty cell above the diagonal is masked. The two strips list the key and the value of every position the model has seen.

Answer A3

10

Which needs more memory?

Course model, 25M parameters.
Serving 32 sequences of 1,024 tokens in BF16:
the weights or the KV cache?

Answer A4

Every KV head costs cache memory

KV cache

$$\text{KV bytes} = 2BSLH_{kv}D_h \text{ bytes/element}$$

- Cached: K, V of every past token
- Grows with batch, context, depth
- One factor to cut: `H_kv`

▼ The cache scales with every factor of the payload

The slide writes the payload of the previous slide in bytes:

$$\text{KV bytes} = 2\,B\,S\,L\,H_{kv}\,D_h \cdot \text{bytes per element}.$$

During decoding, each layer keeps the key and value vectors of every past token, for every sequence in the batch and every KV head. The 2 counts keys and values separately.

Claim. The cache size is proportional to each of its factors: the batch B, the context S, the depth L, the head width D_h, the number of KV heads H_{kv} and the bytes per element of the data type. The formula is a product, so doubling one factor while the others stay fixed doubles the size.

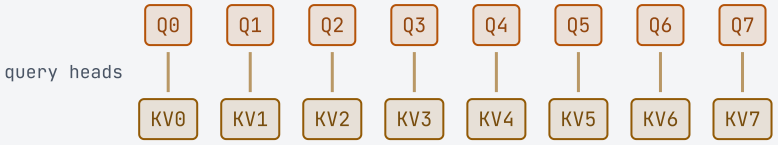
Of these factors, sharing KV heads changes one. It makes H_{kv} smaller, and the cache shrinks in proportion. Whether that saving is worth what the model gives up is a separate question.

The formula counts the cached tensors alone, as an ideal payload, and leaves out the memory traffic of one decode step. A serving system adds paging and allocator metadata, and it may shrink the cache by quantizing it or by keeping only a local attention window. Week 11 covers allocation, eviction and the rest of the cache's lifecycle.

Query heads can share key and value heads

- MHA: `n_kv_heads = n_heads`
- **Grouped-query attention** (GQA): a divisor in between (Ainslie et al., 2023)
- **Multi-query attention** (MQA): `n_kv_heads = 1` (Shazeer, 2019)
- Each KV head serves `g` query heads

ILLUSTRATIVE variant: MHA GQA MQA



query heads	KV heads	KV cache vs MHA
8	8	8/8 (full)

MHA keeps one KV head per query head. MQA shares a single KV head among all query heads. GQA lies between the two: each KV head serves *g* query heads. Fewer KV heads make the KV cache smaller at inference, which Week 11 covers.

▼ Mapping query heads to shared KV heads

Write `n_heads` as H, the number of query heads, and `n_kv_heads` as H_{kv}, the number of key-value heads. The query projection keeps all H heads, while the key and value projections produce only H_{kv} heads. The projected tensors therefore have the shapes `Q:[B,H,S,D_h]` and `K,V:[B,H_kv,S,D_h]`.

Multi-head attention (MHA) is the case $H_{kv} = H$. Multi-query attention (MQA) is the case $H_{kv} = 1$, in which every query head reads one shared key and value head (Shazeer, 2019). Grouped-query attention (GQA) covers the cases in between, with H_{kv} a divisor of H strictly between 1 and H (Ainslie et al., 2023). Every valid configuration has H divisible by H_{kv}, in code `n_heads % n_kv_heads == 0`, and the course configuration rejects any other.

The group map

Definition. Each KV head serves a group of

$$g = \frac{H}{H_{kv}}$$

query heads, and the groups are consecutive. Query head h, counted from zero, reads KV head

$$h \mapsto \lfloor h/g \rfloor, \qquad h = 0, 1, \ldots, H-1.$$

The first g query heads read KV head 0, the next g read KV head 1, and so on. The exercise requires this order. For $H = 4$ and $H_{kv} = 2$ the group size is $g = 2$:

Query head h	0	1	2	3
KV head $\lfloor h/2 \rfloor$	0	0	1	1

MHA is the endpoint $g = 1$, where each group holds one query head and query head h reads KV head h. MQA is the endpoint $g = H$, where one group holds all query heads and every one of them reads KV head 0.

Sharing without copying

The query heads of a group reuse its keys and values logically. An implementation may present one KV head to several query heads, for example through a broadcast view, and the cache still stores that head once. If you expand the KV heads to H heads instead, the expansion has to reproduce the group map. The call `repeat_interleave(g, dim=1)` does: it repeats each KV head g times in place, so expanded head h is KV head $\lfloor h/g \rfloor$ and each group stays consecutive. The call `repeat(1, g, 1, 1)` tiles the whole set of KV heads g times, so expanded head h is KV head $h \bmod H_{kv}$. For $H = 4$ and $H_{kv} = 2$:

Expanded head h	0	1	2	3
`repeat_interleave(2, dim=1)`	0	0	1	1
`repeat(1, 2, 1, 1)`	0	1	0	1

With `repeat`, the shapes are right and the pairs are wrong: query head 1 reads KV head 1, which belongs to query heads 2 and 3. The two calls agree at the endpoints. For MHA, $g = 1$ and both leave the heads as they are. For MQA, $H_{kv} = 1$ and both repeat the one head. A test at MHA or MQA therefore cannot catch the wrong order. Only a configuration strictly between them, such as $H_{kv} = 2$, exposes it.

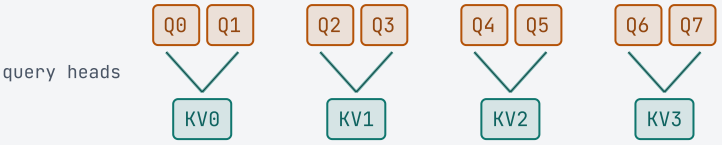
Reading the widget

The widget keeps eight query heads and changes only the number of KV heads they share. Read its states in the order MHA, GQA, MQA. The last column gives the course model's KV heads in the same state:

State	KV heads, $H = 8$	Readout "KV cache vs MHA"	KV heads, $H = 4$
MHA	8	`8/8 (full)`	4
GQA	4	`4/8 of MHA`	2
MQA	1	`1/8 of MHA`	1

The course model has four query heads, so its counts differ. Its GQA case with $H_{kv} = 2$ keeps the same fraction of KV heads as the widget's, $4/8 = 2/4 = 1/2$, and in both GQA cases each KV head serves $g = 8/4 = 4/2 = 2$ query heads. The fractions compare ideal cache payloads with the MHA configuration.

ILLUSTRATIVE variant: MHA GQA MQA



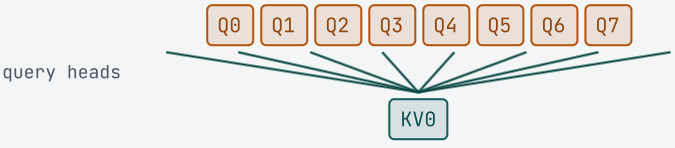
KV heads 4 KV heads

query heads	KV heads	KV cache vs MHA
8	4	4/8 of MHA

MHA keeps one KV head per query head. MQA shares a single KV head among all query heads. GQA lies between the two: each KV head serves *g* query heads. Fewer KV heads make the KV cache smaller at inference, which Week 11 covers.

GQA: each of the 4 KV heads serves 2 consecutive query heads, and the KV cache drops to 4/8 of MHA's.

ILLUSTRATIVE variant: MHA GQA MQA



KV heads 1 KV head (shared)

query heads	KV heads	KV cache vs MHA
8	1	1/8 of MHA

MHA keeps one KV head per query head. MQA shares a single KV head among all query heads. GQA lies between the two: each KV head serves *g* query heads. Fewer KV heads make the KV cache smaller at inference, which Week 11 covers.

MQA: all 8 query heads read one shared KV head, and the KV cache drops to 1/8 of MHA's.

What GQA Saves

- `D=512`, `H=4`, `D_h=128`, `L=8`, `S=1024`, BF16

Variant	projection params / layer	cache `B=1`	cache `B=32`
MHA (`H_kv=4`)	1,048,576	16 MiB	512 MiB
GQA (`H_kv=2`)	786,432	8 MiB	256 MiB
MQA (`H_kv=1`)	655,360	4 MiB	128 MiB

- QK and AV work unchanged
- Speed depends on the implementation
- Quality: compare loss curves (Ainslie et al., 2023)

▼ Reproducing the parameter, cache and work counts

The table holds the course base configuration fixed and changes only the number of KV heads, through $H_{kv} = 4$, 2 and 1.

Assumptions

- **A1. The course base configuration.** The model width is $D = 512$, with $H = 4$ query heads of width $D_h = 128$, so $D = HD_h$. The model has $L = 8$ layers and sees sequences of $S = 1{,}024$ tokens. These are the values on the slide.
- **A2. Bias-free dense projections.** W_Q and W_O map width D to width D, and W_K and W_V map width D to width $H_{kv}D_h$. The course's projections carry no bias, so every parameter is a matrix entry.
- **A3. BF16 storage.** Each cached element takes 2 bytes, as on the previous slides, and $1\,\mathrm{MiB} = 2^{20}$ bytes.
- **A4. The MLP width.** The block's MLP has two matrices, $D \to D_{ff} \to D$, with $D_{ff} = 3D = 1{,}536$ as in Week 2. Only the work count uses it.

Parameters per layer

Claim. Under A1 and A2, the attention projections of one layer hold

$$P_{\text{attn}}(H_{kv}) = 2D^2 + 2D\,H_{kv}\,D_h \tag{1}$$

parameters. Each projection matrix holds one parameter per entry:

$$P_{\text{attn}}(H_{kv}) = \underbrace{D \cdot D}_{W_Q} + \underbrace{D \cdot D}_{W_O} + \underbrace{D \cdot H_{kv} D_h}_{W_K} + \underbrace{D \cdot H_{kv} D_h}_{W_V} \quad \text{A2}$$
$$= 2D^2 + 2D\, H_{kv}\, D_h.$$

The Q and O term, $2D^2 = 2 \cdot 512^2 = 524{,}288$, is the same in every variant. The K and V term is proportional to H_{kv}:

Variant	H_{kv}	Q and O, $2D^2$	K and V, $2D\, H_{kv}\, D_h$	P_{attn}
MHA	4	524,288	524,288	1,048,576
GQA	2	524,288	262,144	786,432
MQA	1	524,288	131,072	655,360

For MHA, $H_{kv} D_h = H D_h = D$, and equation (1) gives back the $4D^2$ of the Multi-Head Attention tab.

Over the whole model, the saving is the change in the K and V term times the $L = 8$ layers. For MQA it is $8 \cdot (524{,}288 - 131{,}072) = 3{,}145{,}728$ parameters, so the model drops from 25,174,528 to 22,028,800 parameters, about 12.5% fewer.

Cache bytes

Claim. Under A1 and A3, at a fixed batch and context the cache is proportional to H_{kv}, and sharing KV heads divides it by the group size $g = H / H_{kv}$.

Compare the payload of "Weights against cache in one serving setup" with its MHA value:

$$\frac{2\, B\, S\, L\, H_{kv}\, D_h}{2\, B\, S\, L\, H\, D_h} = \frac{H_{kv}}{H} = \frac{1}{g}.$$

Every other factor cancels. For one sequence of the MHA model, the payload in bytes is

$$2 \cdot 1 \cdot 1024 \cdot 8 \cdot 4 \cdot 128 \cdot 2 = 16{,}777{,}216,$$

which is 16 MiB. At $B = 32$ each payload is 32 times larger:

Variant	H_{kv}	Bytes, $B = 1$	MiB, $B = 1$	MiB, $B = 32$
MHA	4	16,777,216	16	512
GQA	2	8,388,608	8	256
MQA	1	4,194,304	4	128

The work of one block

Definition. One scalar multiply-accumulate counts as one MAC. The count covers one block's forward pass: the bias-free Q, K, V and O projections, the two $S \times S$ matrix products of attention, and the two MLP products. It takes both attention products in full, even though the causal mask blocks the upper triangle. It leaves out embeddings, normalization, RoPE, building and applying the mask, softmax, the activation, residual additions, dropout, cache traffic, the backward pass, optimizer work and every kernel or backend effect.

The two attention products are the scores $QK^\top$ and the weighted sum AV, where A holds the attention weights of shape `[B,H,S,S]`. One fact from linear algebra is used: multiplying an $m \times n$ matrix by an $n \times p$ matrix takes

$$mnp \text{ MACs.} \tag{F1}$$

Claim. Under A1, A2 and A4, the projections, the attention products and the MLP of one block take

$$\begin{aligned}
M_{\text{proj}} &= BS\,P_{\text{attn}}(H_{kv}) && \text{F1} \\
&= BS\,(2D^2 + 2D\,H_{kv}\,D_h) && (1) \\
M_{\text{attn}} &= BH\,(S^2 D_h + S^2 D_h) && \text{F1, two products per head} \\
&= 2BHS^2 D_h \\
&= 2BS^2 D && \text{A1: } HD_h = D \\
M_{\text{MLP}} &= BS\,(D\,D_{ff} + D_{ff}\,D) && \text{F1, A4} \\
&= 2BS\,D\,D_{ff}
\end{aligned}$$

MACs. The projections take one MAC per weight and token, each head computes the two products $QK^\top$ and AV, and the MLP has two matrices. Only the projection term contains H_{kv}. At $B = 1$:

Variant	M_{proj}	M_{attn}	M_{MLP}	Total
MHA	1,073,741,824	1,073,741,824	1,610,612,736	3,758,096,384
GQA	805,306,368	1,073,741,824	1,610,612,736	3,489,660,928
MQA	671,088,640	1,073,741,824	1,610,612,736	3,355,443,200

The QK and AV work stays fixed because H, S and D_h stay fixed. Every query head still forms its own scores and its own weighted sum of values, so the logical score tensor is still `[B,H,S,S]`, with the same softmax over the same entries. Fewer KV heads reduce the K and V projection work and the cache bytes. To report nominal FLOPs, with one fused multiply-add counted as two FLOPs, double each MAC total.

Speed and quality are measured

Real kernels may handle the grouping and the memory traffic differently. End-to-end speed therefore depends on the setting and the implementation, and any speedup has to be measured.

Quality needs its own evidence.

Reported. Ainslie et al. found quality close to MHA for GQA in their setup of uptrained T5 models (Ainslie et al., 2023). At course scale you observe quality only as a training-loss comparison at matched steps and tokens, and Week 3 runs no task evaluation. The exercise's `kv_heads` comparison does this. It sets the loss curves and tokens per second of the `mha-rope`, `gqa-2` and `mqa` arms against their K and V parameters and cache bytes.

GQA in released models

GQA is a common choice in released decoders, but not a universal default. Llama 3 uses GQA with 8 KV heads at all three of its sizes, to improve inference speed and to reduce the size of the KV cache during decoding (Grattafiori et al., 2024). OLMo 3 uses MHA at 7B, with 32 query and 32 KV heads, and GQA at 32B, with 40 query and 8 KV heads (Ettinger et al., 2025). Week 4 keeps this divisible `n_kv_heads` configuration.

Causal Masking in Depth

14

One rule decides every allowed pair

$$\begin{aligned}\text{allowed}[b, q, k] = \text{valid_query}[b, q] \wedge (k \leq q) \\ \wedge \text{valid_key}[b, k] \wedge \text{same_document}[b, q, k]\end{aligned}$$

▼ A pair is allowed only when all four terms hold

Week 2 needed one mask, the causal one. Real batches add two complications: sequences of different lengths are padded to a common length, and several short documents are packed into one row. Both change which query-key pairs may interact, so the course writes one rule that covers all of them.

Definition. The sequence index b selects a sequence in the batch, the query row q is the position whose output is being computed, and the key column k is a candidate position to read from. In this tab, q and k index rows and columns of the score matrix, not query and key vectors. Whether the pair (q, k) of sequence b is allowed is the conjunction of four Boolean terms:

$$\begin{aligned}\text{allowed}[b, q, k] = \text{valid_query}[b, q] \wedge (k \leq q) \\ \wedge \text{valid_key}[b, k] \wedge \text{same_document}[b, q, k].\end{aligned}$$

Each of the four terms asks one question:

- the term $\text{valid_query}[b, q]$ holds when the query is a real token that may produce an attention output;
- the condition $k \leq q$ keeps the diagonal and blocks every future key, which is the causal mask of Week 2;
- the term $\text{valid_key}[b, k]$ holds when the key is a real source token;
- the term $\text{same_document}[b, q, k]$ holds when the query and the key belong to the same document.

The rule defines which pairs may interact. It says nothing about how a kernel computes them.

A pair is allowed only when all four terms hold: a real query, a key that is not in its future, a real key, and the same document. The two validity terms handle padding, the causal term is Week 2's mask, and the document term handles packing. All three are parts of one permission rule. A

conjunction does not depend on the order of its terms, so there is no precedence between separate masks to get right: there is only one mask.

SDPA takes one mask, and True means allowed Read on your own

Case	scaled dot-product attention (SDPA) call
plain causal	`attn_mask=None`, `is_causal=True`
padding or packing	one combined explicit mask, `is_causal=False`
additive mask	`0` allowed, `-inf` blocked

- `nn.MultiheadAttention`: `True` blocks (PyTorch Contributors, 2026c)
- Never a mask with `is_causal=True` (PyTorch Contributors, 2026d)

▼ **Passing the mask to SDPA with the right polarity**

The exercise computes attention with PyTorch's scaled dot-product attention (SDPA), so the permission rule has to reach SDPA in the form SDPA expects.

Build the mask once, broadcastable to the shape of the attention scores. For scores of shape `[B,H,S_q,S_k]` the mask is typically `[B,1,S_q,S_k]`, where B counts sequences, H query heads, S_q query positions and S_k key positions. Put the mask on the same device as the attention inputs.

SDPA accepts the mask in two forms. A boolean mask is the rule itself: `True` means allowed and `False` means blocked (PyTorch Contributors, 2026d). An additive mask M is added to the scaled scores before the softmax, as in Week 2, with

$$M[b,q,k] = \begin{cases} 0 & \text{if allowed}[b,q,k], \\ -\infty & \text{otherwise.} \end{cases}$$

Build or cast the additive mask in a floating dtype the scores accept, on their device.

The module `nn.MultiheadAttention` uses the opposite polarity: in its boolean masks, `True` means blocked (PyTorch Contributors, 2026c). A boolean mask written for that module therefore cannot be passed to SDPA unchanged.

Plain causal attention may use `attn_mask=None` with `is_causal=True`. Any case with padding or packing passes the one combined explicit mask with `is_causal=False`.

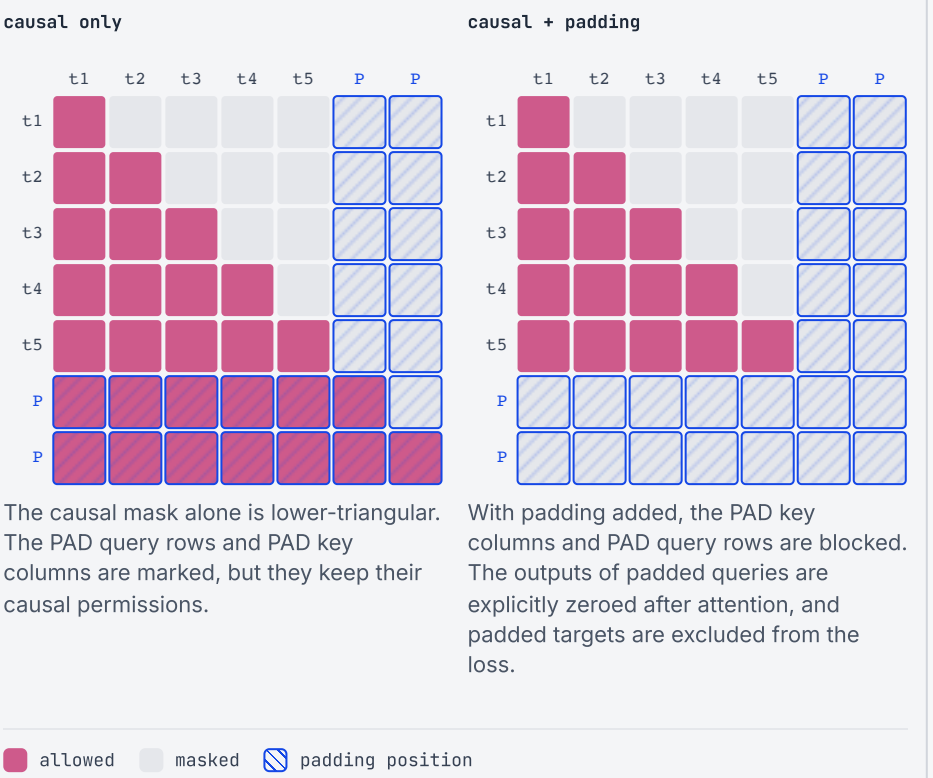
Never pass an explicit mask together with `is_causal=True`. The SDPA documentation for the target version describes the two as mutually exclusive (PyTorch Contributors, 2026d). A check on 2026-07-14 found that the CPU math path accepted both arguments and composed them,

and the CUDA backend the course selects has not been checked. The course rule, one explicit mask whenever there is one, holds whether a backend accepts the combination or rejects it.

Padding blocks keys, queries and targets Read on your own

- Block padded key columns
- Zero padded-query outputs after attention
- Skip padded targets in the loss

ILLUSTRATIVE PADDING



The causal mask alone is lower-triangular. The PAD query rows and PAD key columns are marked, but they keep their causal permissions.

With padding added, the PAD key columns and PAD query rows are blocked. The outputs of padded queries are explicitly zeroed after attention, and padded targets are excluded from the loss.

▼ Padding: block the keys, zero the outputs, skip the targets

Padding needs handling in three places: the key columns, the query rows and the targets.

Reading the figure

Both grids have one row per query and one column per key, over five real tokens t1 to t5 and two padding positions P. The grid "causal only" applies $k \leq q$ alone, so the first padded query may still read every earlier column. The grid "causal + padding" adds the two validity terms. The term $\mathrm{valid_key}$ blocks the P key columns, so no real query reads padding. The term $\mathrm{valid_query}$ blocks the P query rows: a padded query has no allowed key at all.

The output and the loss are separate steps

The mask decides which values attention may combine. It defines neither the final output nor the training target, so two more steps follow the attention call.

First, zero every padded query's attention output explicitly. Let $O[b, q]$ be the attention output of query row q. Replace the padded rows, with `torch.where` or `masked_fill`:

$$O[b, q] \leftarrow \begin{cases} O[b, q] & \text{if valid_query}[b, q], \\ 0 & \text{otherwise.} \end{cases}$$

Do not multiply the output by a 0/1 mask instead. A NaN the backend produced survives the multiplication, because $0 \cdot \mathrm{NaN} = \mathrm{NaN}$, while a replacement discards the old value.

Replacing the rows cleans the output but not the gradient. The gradient that reaches a replaced row is zero, but the backward pass of attention multiplies it by the derivatives of the NaN row, and $0 \cdot \mathrm{NaN} = \mathrm{NaN}$. The backend itself must therefore return finite rows.

Second, leave out of the shifted next-token loss every position whose target is padding. In this sequence that includes the step from the last real token t5 to the first P.

The output zeroing and the loss masking act on different tensors, so test each of them on its own.

A fully blocked row needs a policy Read on your own

- Test boolean and additive masks
- Rows finite and exactly zero
- Hand-written softmax: one sentinel key

▼ Why a fully blocked row needs an explicit policy

Definition. Query row q of sequence b is **fully blocked** when $\text{allowed}[b, q, k]$ is false for every key column k. A padded query row is fully blocked, because $\text{valid_query}[b, q]$ is false, and that one false term makes the conjunction false for every key column k.

The ordinary softmax divides zero by zero

Fix one sequence and one head. Write $u_{q,k}$ for the finite scaled score of query row q against key column k, and $a_{q,k}$ for its attention weight.

- **A1. An additive mask.** The mask enters as $M[b, q, k] = -\infty$ for every blocked pair, the form SDPA accepts as an additive mask.
- **A2. The ordinary softmax.** The weights are Week 2's softmax over the key columns, with no special case for an empty row.

One fact is recalled from Week 2:

$$e^{-\infty} = 0. \tag{F1}$$

Claim. Under A1 and A2, every weight of a fully blocked row is the undefined quotient $0/0$.

$$\begin{aligned} z_{q,k} &= u_{q,k} + M[b, q, k] = -\infty && \text{A1, every } k \text{ blocked} \\ e^{z_{q,k}} &= 0 && \text{F1} \\ \textstyle\sum_l e^{z_{q,l}} &= 0 && \text{a sum of zeros} \\ a_{q,k} &= \frac{e^{z_{q,k}}}{\sum_l e^{z_{q,l}}} = \frac{0}{0} && \text{A2} \end{aligned}$$

In floating point, this $0/0$ can produce NaN (PyTorch Contributors, 2026b). The weights of a row must sum to one over its allowed keys, and a fully blocked row has none, so the ordinary softmax has no valid answer for it.

Test the backend

What SDPA returns for such a row depends on the backend, and the mechanism above does not decide it. Test fully blocked rows yourself, with a boolean and with an additive mask, on your device, dtype and selected backend. Require the outputs to be finite, and the padded

rows to be exactly zero after the output zeroing. The guarantee is specific to the backend, so you observe it in a test.

Finite stand-ins for $-\infty$

Some code writes a large finite negative number in place of $-\infty$, for example the most negative value of the score dtype, `torch.finfo(dtype).min`. For a row with at least one allowed key, the result is the same in practice: a blocked score lies so far below the allowed ones that its exponential underflows to 0 once the softmax subtracts the row maximum. A fully blocked row then gives no $0/0$, because every score is finite. Its blocked scores are all about equal, so the softmax spreads its weight over keys the row may not read, and the output is a finite but meaningless average. A finite stand-in therefore replaces NaN by a wrong value, and the output zeroing and the loss masking are still needed.

Two cautions apply. The stand-in must be representable in the dtype of the scores: -10^9, for example, lies far below the most negative FP16 value, about $-65{,}504$, and becomes $-\infty$ in FP16. And, as Week 2's note on scaling before masking showed, a stand-in added before the division by $\sqrt{d_k}$ shrinks with it.

The sentinel key for a hand-written softmax

Code with a hand-written softmax path needs one more rule. If that path cannot represent an empty row safely, temporarily allow one sentinel key, the query's own position $k = q$. The denominator is then $e^{u_{q,q}}$, which is finite and positive, so the softmax puts weight 1 on that key, and the output is the value vector at the padded position itself.

The sentinel is a numerical guard and grants no real permission. Zero that query's output explicitly after attention, so its arbitrary attended value cannot reach later computation, and leave its padded or boundary target out of the loss. This week's exercise has no hand-written softmax path, so the sentinel rule applies only to code that has one.

The four safeguards are independent: the backend test, the sentinel, the output zeroing and the loss masking. Each guards against a different failure, and none of them replaces another.

18

Which keys may B2 read?

Packed row: A1 A2 A3 B1 B2 B3

Answer A5

A sparse mask can still run dense Read on your own

- The mask sets allowed pairs
- Kernels may still compute every score
- Block skipping: Week 4 (PyTorch Contributors, 2026a)

▼ **Allowed pairs and executed work are separate questions**

The allowed-pair pattern answers one question: which key values may influence each query. Executed work is a second question. With a dense explicit mask, a backend can compute every score, apply the mask or bias before the softmax, and then compute the full product of probabilities and values. The number of blocked cells therefore says nothing about speed.

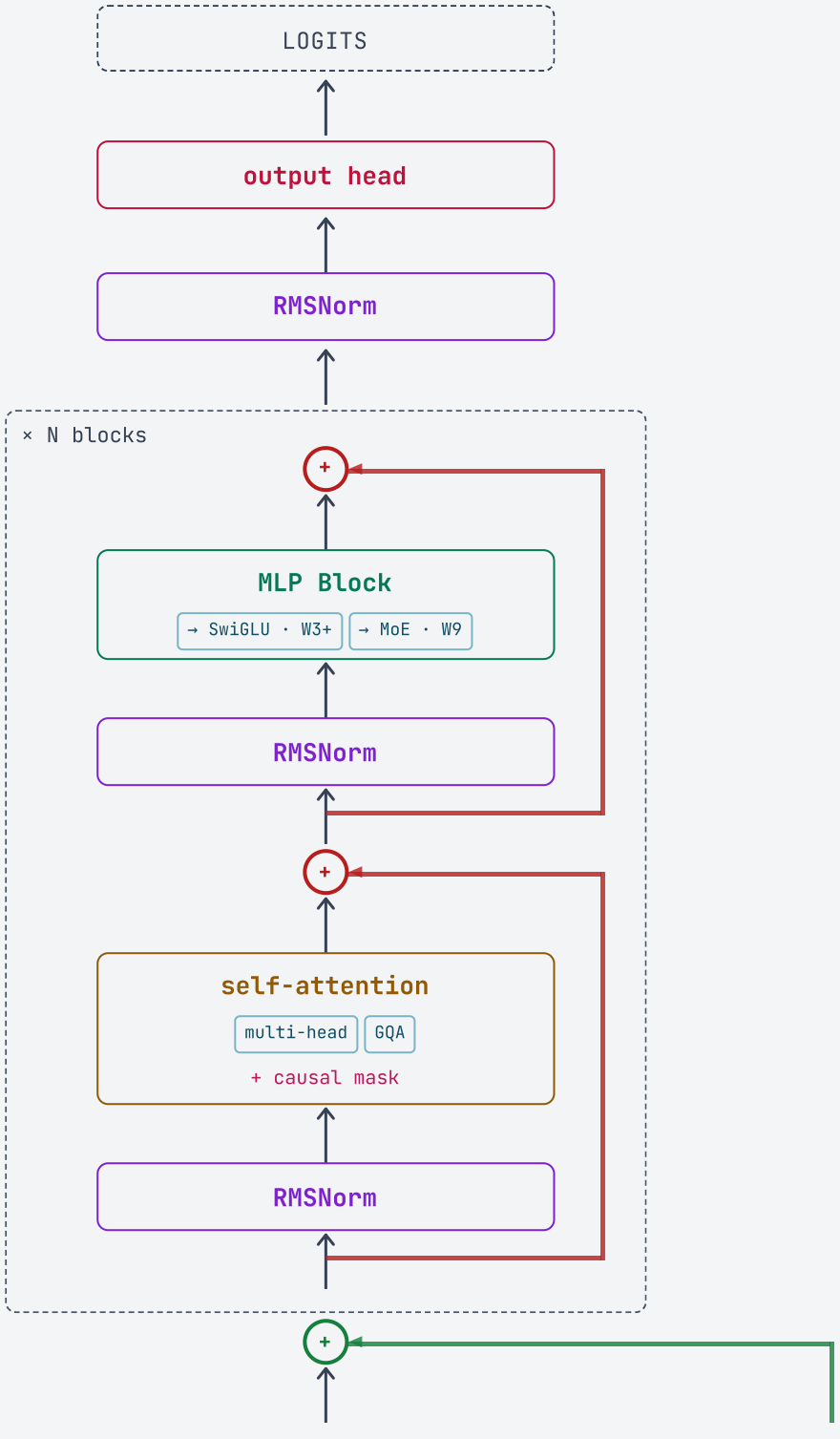
FlexAttention shows that block structure can be represented explicitly (PyTorch Contributors, 2026a). Week 4 covers how block representations map to skipped work, the I/O behavior of FlashAttention, and measured performance.

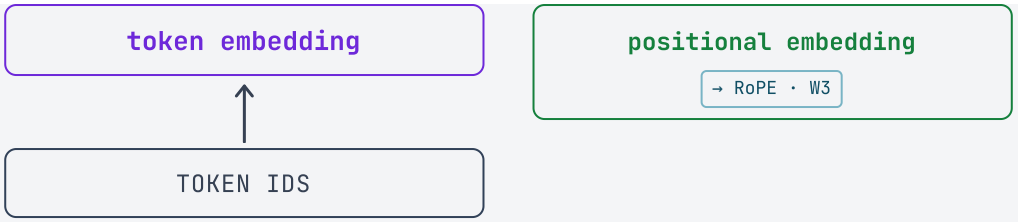
The Full Stack

20

Repeat the block, then normalize once

- Same block, `L` times, `[B,S,D]` throughout
- Final RMSNorm, then the tied head
- Later layers build on earlier ones





▼ From one block to a stack

One block performs one attention update and one MLP update. A language model needs a sequence of such transformations, so that representations can be refined step by step and circuits can compose across layers. Week 2 built the computation inside one block. Repeating that block changes what each block receives. Block 2 reads the residual stream that block 1 produced, block 3 reads the result of block 2, and so on. Attention in one layer can route information that a later MLP transforms, and a still later attention update can route the transformed features again. The induction pattern is one example of such composition. Depth makes cross-layer composition possible. It does not make every useful head an induction head. Training decides which circuits appear, and no particular one is guaranteed.

One residual stream through the stack

Let B be the batch size, S the sequence length, D the model width and L the number of blocks, which the diagram labels `N`. Write x_ℓ for the residual stream of shape `[B,S,D]` that enters block ℓ, and a_ℓ for the stream after that block's attention update. **Definition.** Each block applies two pre-norm residual updates:

pre-norm block

$$a_\ell = x_\ell + \mathrm{Attention}(\mathrm{RMSNorm}(x_\ell))$$
$$x_{\ell+1} = a_\ell + \mathrm{MLP}(\mathrm{RMSNorm}(a_\ell))$$

Every term has shape `[B,S,D]`. The attention output projection returns to width D, and the MLP widens each position to D_{ff} and projects it back to D. Applying the two updates for $\ell = 0, \ldots, L-1$ carries one residual stream through all L blocks, from x_0 to x_L. Initialization, and how the residual stream grows with depth, are covered in Week 5.

The final norm and the tied head

Let V be the vocabulary size and E the `[V,D]` token-embedding matrix, which the tied output head reuses. After the last block, one final RMSNorm gives the stream z, still of shape `[B,S,D]`, and the tied projection gives logits of shape `[B,S,V]`:

$$z = \mathrm{RMSNorm}(x_L), \qquad \mathrm{logits} = zE^\top.$$

The final RMSNorm is outside the repeated block, so the stack applies it once, while each block has two RMSNorms of its own. A stack of L blocks therefore has $2L + 1$ RMSNorms.

Reading the diagram

The diagram shows the Week 3 stack with every stage visible. Read it from the bottom up. Each labeled element opens a short description of its role. The arrows show which values depend on which. A runtime schedule, the order in which kernels run, a parallelism strategy and the circuits a trained model learns are not part of the picture.

SwiGLU, the MLP variant the diagram points to

The diagram's MLP node carries a chip that points to SwiGLU, a gated variant of the MLP (Shazeer, 2020). The course MLP from Week 2 has two matrices, $D \to 3D \to D$, with GELU between them. SwiGLU adds a third matrix whose output gates the hidden vector. Three matrices of hidden width D_{ff}^{gated} hold as many parameters as two of hidden width D_{ff} when $D_{ff}^{\text{gated}} = \frac{2}{3} D_{ff}$. Shazeer shrinks the hidden width by this factor of 2/3 to keep the parameter count and the computation the same. The Week 2 reading slide "Gated MLPs multiply two projections" derives this and reports what Shazeer found.

Many recent models use SwiGLU. Llama 3 does (Grattafiori et al., 2024), and so do Qwen3 (Yang et al., 2025a) and OLMo 3 (Ettinger et al., 2025). DeepSeek-V3 uses it in its mixture-of-experts (MoE) layers (DeepSeek-AI, 2024), and gpt-oss in its experts (OpenAI, 2025). Gemma 3 uses the GELU-gated form, GeGLU (Google, 2025). The ungated two-matrix form remains in use as well. The GPT lineage uses it with GELU (OpenAI, 2019), and nanochat with a squared ReLU (Karpathy, 2026). The course keeps the two-matrix GELU MLP, and the exercise uses it unchanged.

The course recipe sets width and heads from depth Read on your own

depth recipe $D = 64L, \quad H = D/128, \quad D_{ff} = 3D$

L	D	H	D_ff	tied parameters
4	256	2	768	4.7M
8	512	4	1536	25.2M
12	768	6	2304	77.1M
16	1024	8	3072	176.2M

▼ Depth, width and the parameter count

Heuristic. In this example recipe, depth is the input: you choose the number of blocks L, and the recipe sets the model width D, the head count H and the MLP width D_{ff} from it. The rule for the width and the heads comes from nanochat (Karpathy, 2026), which computes

$$D = \left\lceil \frac{64L}{128} \right\rceil \cdot 128, \qquad H = \frac{D}{128}.$$

The course pairs the rule with its own MLP width, $D_{ff} = 3D$. The recipe is a heuristic for choosing example configurations, and it makes no claim about how loss scales. The course's model configuration, `ModelConfig`, does not compute this recipe.

Even and odd depths

Claim. For every even L, the rule gives $D = 64L$ and $H = L/2$. For odd L, it rounds $64L$ up to the next multiple of 128, which is $64(L+1)$.

The fraction inside the ceiling simplifies to $64L/128 = L/2$. For even L this is already an integer, so the ceiling leaves it unchanged:

$$\begin{aligned} D &= \left\lceil \tfrac{L}{2} \right\rceil \cdot 128 \\ &= \tfrac{L}{2} \cdot 128 \qquad L \text{ even, so } L/2 \text{ is an integer} \\ &= 64L, \\ H &= \tfrac{D}{128} = \tfrac{L}{2}. \end{aligned}$$

For odd L, the ceiling rounds $L/2$ up to $(L+1)/2$, and the rule gives $D = 64(L+1)$. Either way D is a multiple of 128, so H is an integer and every head has width $D_h = D/H = 128$. The slide's formula $D = 64L$ is the rule at even depths, and the table samples only even depths. Along the recipe, a deeper model is also a wider one.

The tied parameter count

The parameter column counts the course architecture at the recipe's widths. It is not nanochat's parameter count: the recipe takes only nanochat's rule for the width and the heads, and the blocks are the course's. Let P be the number of trainable parameters. The count rests on five assumptions, each a property of the course model.

- **A1. Tied embeddings.** One matrix E of shape `[V,D]`, with $V = 8192$, is both the token lookup and the output head. This is Week 2's weight tying.
- **A2. Attention.** Each block has bias-free multi-head attention with four square $D \times D$ projections W_Q, W_K, W_V and W_O. The Multi-Head Attention tab showed that these hold $4D^2$ parameters whatever the head count.
- **A3. MLP.** Each block has a bias-free two-matrix GELU MLP, $D \to D_{ff} \to D$. This is the MLP the course keeps.
- **A4. Gains.** Each block has two RMSNorm gains of length D, and the stack has one more after its last block.
- **A5. Nothing else.** The model has no other trainable parameters.

Claim. Under A1 to A5, the tied parameter count is the exact identity

$$P = VD + L\left(4D^2 + 2D\,D_{ff}\right) + (2L+1)D. \tag{1}$$

Count each part on its own:

$$\begin{aligned}
P_{\text{emb}} &= VD && \text{A1: counted once, the head reuses } E \\
P_{\text{attn}} &= 4D^2 && \text{A2: per block} \\
P_{\text{MLP}} &= D\,D_{ff} + D_{ff}\,D && \text{A3: per block} \\
&= 2D\,D_{ff} \\
P_{\text{norm}} &= 2LD + D = (2L+1)D && \text{A4: all blocks and the final gain.}
\end{aligned}$$

By A5 these are all the parameters, so $P = P_{\text{emb}} + L\,(P_{\text{attn}} + P_{\text{MLP}}) + P_{\text{norm}}$, which is (1). It is Week 2's formula (9), there written with d_{ff} for the MLP width and evaluated for one block.

The four configurations

With $D_{ff} = 3D$, the two per-block terms add up to $4D^2 + 6D^2 = 10D^2$, and (1) becomes

$$P = VD + 10LD^2 + (2L+1)D.$$

At the recipe's four depths, with $V = 8192$, the three terms and their sums are:

L	D	VD	$10LD^2$	$(2L+1)D$	P
4	256	2,097,152	2,621,440	2,304	4,720,896
8	512	4,194,304	20,971,520	8,704	25,174,528
12	768	6,291,456	70,778,880	19,200	77,089,536
16	1,024	8,388,608	167,772,160	33,792	176,194,560

The slide's table rounds the totals to millions. The row $L = 8$ has the widths of the course configuration, $D = 512$ and $H = 4$, and its total is the 25,174,528 that Week 2 computed for eight blocks.

Depth alone

Moving from one row to the next changes L, D, H and D_{ff} together, so a comparison between rows cannot isolate the effect of depth. To vary depth alone, hold D, H and D_{ff} fixed.

Claim. At fixed D, H and D_{ff}, each extra block adds $4D^2 + 2D\,D_{ff} + 2D$ parameters.

Separate the terms of (1) that do not grow with L from those that do:

$$\begin{aligned} P &= VD + L\left(4D^2 + 2D\,D_{ff}\right) + 2LD + D && \text{expand } (2L+1)D \\ &= (V+1)D + L\left(4D^2 + 2D\,D_{ff} + 2D\right) && \text{collect the terms in } L. \end{aligned}$$

The first part, the tied embedding and the final gain, is paid once, because neither repeats. The second part is one copy of the per-block count for each block, so one more block adds one more copy. Parameter counts alone do not tell you how stably the model trains or how good it becomes.

Rotary Position Embeddings

The score should depend on the offset

- Score compares query `m` with key `n`
- Language structure mostly follows `n - m`
- Sinusoidal: added once, mixed with content
- Nothing forces the score onto `n - m`

▼ **Position in the residual stream is a weaker guarantee than position in the score**

Attention scores a query at position m against a key at position n. Much of the structure of language depends on the offset $n - m$ between the two tokens, so a useful position signal should reach the score as that offset.

In Week 2, sinusoidal position entered the model once, before the first block, as an elementwise addition to the token embedding. That puts positional information into the residual stream. The sinusoidal basis also carries relative structure: Week 2 showed that shifting a position by a fixed offset acts linearly on each frequency pair, as a rotation by an angle that depends on the offset and the pair but not on the position. The two facts are often read as a guarantee that the score sees the offset. Neither gives one.

To see why, write out the score of one head. Let $u_m, u_n \in \mathbb{R}^D$ be the normalized hidden states at positions m and n that enter the attention branch, and let $W_Q^{(h)}, W_K^{(h)} \in \mathbb{R}^{D \times D_h}$ be the query and key projections of head h. The head computes $q_m = W_Q^{(h)\top} u_m$ and $k_n = W_K^{(h)\top} u_n$, and its score before the $1/\sqrt{D_h}$ scale is

$$q_m^\top k_n = u_m^\top W_Q^{(h)} W_K^{(h)\top} u_n.$$

This is a bilinear form in the two hidden states. Each state mixes a token component and a position component in the same D channels, and each depends on its own position, m or n. Nothing in the form makes the score depend on $n - m$ instead of on m and n separately. The model has to learn to recover the offset through W_Q and W_K, and whether it can depends on what those projections learn. Every layer that rewrites the residual stream can also weaken or overwrite the position component it inherited.

Make q·k depend only on n - m.

Rule: q may use only m, and k only n.

- Work in 2D: q and k as arrows
- What sets `q·k`: lengths, angle, both?
- What turns an arrow, keeps its length?

Rotating q and k leaves only the offset

column-vector rotation convention

$$R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

relative-rotation derivation

$$R(m\omega)^\top R(n\omega) = R(-m\omega)R(n\omega) = R((n-m)\omega)$$

relative-score identity

$$(R_m q)^\top (R_n k) = q^\top R_{n-m} k, \quad R_m = R(m\omega)$$

- Rotate Q and K, never V
- Full head: all `D_h` coordinates, in pairs
- Holds for one score term (Su et al., 2024)

▼ Deriving the relative-score identity

The usual ideas

The task asks for a change to q that uses only the query position m, and a change to k that uses only the key position n, such that their dot product depends on the two positions only through $n - m$. Four ideas usually come up.

- **Add the offset to the score.** The rule forbids it, because q does not know n and k does not know m.
- **Scale by powers of a constant.** Multiply q by c^m and k by c^{-n} for some constant c. The product is $(c^m q)^\top (c^{-n} k) = c^{m-n}\, q^\top k$, so it depends on the positions only through the offset, and the idea is correct. But the lengths of the vectors, $c^m|q|$ and $c^{-n}|k|$, then grow or shrink with position, while a rotation keeps them.
- **Rotate.** Turn each vector by an angle proportional to its position.
- **Use a complex phase.** Read each two-dimensional vector as a complex number, and multiply q by $e^{im\omega}$ and k by $e^{in\omega}$. Multiplying by $e^{i\theta}$ turns a complex number by the angle θ and keeps its modulus, so this is the same rotation in other notation.

The rotation is the idea RoPE uses. It leaves the token content alone and rotates q and k after the head reshape. The offset then appears in the score by construction, as a property of the rotation, instead of as something the projections must learn to expose. RoPE changes where position enters the computation, and it leaves the amount of position information as it was.

Assumptions and claim

Take one pair of coordinates of one head first.

- **A1. Fixed unrotated vectors.** The vectors $q, k \in \mathbb{R}^2$ are one coordinate pair of the query at position m and of the key at position n, as the projections and the head reshape produce them. The derivation holds them fixed, so it concerns one score term.
- **A2. One frequency for both.** The query pair and the key pair turn at the same fixed angular frequency ω, in radians per position. RoPE takes the frequency from the pair index alone, so queries and keys share it.
- **A3. Angle proportional to position.** The query is rotated by $R_m = R(m\omega)$ and the key by $R_n = R(n\omega)$, where $R(\theta)$ is the counter-clockwise rotation on the slide, acting on column vectors.

Claim (exact identity). Under A1 to A3,

$$(R_m q)^\top (R_n k) = q^\top R_{n-m}\, k, \qquad R_{n-m} = R\big((n-m)\omega\big). \tag{1}$$

The positions enter the score only through the offset $n - m$. The identity is derived twice below, first as a picture and then in matrices.

The picture in two dimensions

Write $|q|$ and $|k|$ for the lengths of the two arrows, and φ_q and φ_k for their angles from the first axis, so that $q = |q|\,(\cos\varphi_q, \sin\varphi_q)^\top$ and $k = |k|\,(\cos\varphi_k, \sin\varphi_k)^\top$. The angle-difference identity of the cosine gives their dot product:

$$q^\top k = |q|\,|k|\,(\cos\varphi_q\cos\varphi_k + \sin\varphi_q\sin\varphi_k) = |q|\,|k|\cos(\varphi_k - \varphi_q).$$

The dot product depends on the two lengths and on the angle between the arrows. A rotation by θ keeps the length of a vector and adds θ to its angle. After both rotations the lengths are unchanged and the angles are $\varphi_q + m\omega$ and $\varphi_k + n\omega$, so their difference gains $n\omega - m\omega$:

$$(R_m q)^\top (R_n k) = |q|\,|k|\cos\big(\varphi_k - \varphi_q + (n-m)\omega\big).$$

The absolute angles $m\omega$ and $n\omega$ enter only through their difference, and only the offset is left.

The identity in matrices

The matrix version needs two facts about rotations. The first follows because the cosine is even and the sine is odd, the second from the angle-addition identities:

$$R(\theta)^\top = R(-\theta) \tag{F1}$$

and

$$R(a)\,R(b) = R(a+b). \tag{F2}$$

So the transpose of a rotation matrix is the rotation by the opposite angle, which undoes it, and two rotations compose by adding their angles. With these facts,

$$\begin{aligned}
(R_m q)^\top (R_n k) &= q^\top R(m\omega)^\top R(n\omega)\, k && \text{transpose of a product, A3}\\
&= q^\top R(-m\omega)\, R(n\omega)\, k && \text{F1}\\
&= q^\top R\big((n-m)\omega\big)\, k && \text{F2}\\
&= q^\top R_{n-m}\, k && \text{definition of } R_{n-m}.
\end{aligned}$$

The middle steps are the relative-rotation derivation on the slide. The two absolute rotations meet in the product $R_m^\top R_n$, and that product depends on m and n only through $n-m$.

A worked pair

Take $q = (1, 2)^\top$, $k = (3, 4)^\top$, $m = 2$, $n = 5$ and $\omega = 0.1$, so that $m\omega = 0.2$, $n\omega = 0.5$ and $(n-m)\omega = 0.3$. Rotating each vector by its absolute angle and rotating the key once by the offset angle give the same score, up to the rounding of the coordinates:

$$\begin{gathered}
(R(0.2)\,q)^\top (R(0.5)\,k) \approx (0.5827,\ 2.1588)\cdot(0.7150,\ 4.9486) \approx 11.0997,\\
q^\top R(0.3)\,k \approx (1,\ 2)\cdot(1.6839,\ 4.7079) \approx 11.0997.
\end{gathered}$$

Multiplying out gives the exact value. For any angle θ, with the two coordinates of a pair numbered 1 and 2,

$$q^\top R(\theta)\,k = (q_1k_1 + q_2k_2)\cos\theta + (q_2k_1 - q_1k_2)\sin\theta,$$

which for this pair is $11\cos\theta + 2\sin\theta$. Both sides of (1) therefore equal

$$\begin{aligned}
(R(0.2)\,q)^\top (R(0.5)\,k) &= q^\top R(0.3)\,k\\
&= 11\cos 0.3 + 2\sin 0.3 \approx 11.099741793704.
\end{aligned}$$

Shifting both positions by the same amount leaves the score unchanged. At $m = 7$ and $n = 10$ the offset is still 3, and $(R(0.7)\,q)^\top (R(1.0)\,k) \approx 11.0997$ as well.

The full head is a sum over pairs

A head has D_h coordinates, more than two. Week 3 rotates the full head: all D_h coordinates are rotated, in $D_h/2$ pairs, so D_h must be even. Write $q^{(j)} = (q_{2j}, q_{2j+1})^\top$ for pair j of the

query, with $j = 0, \ldots, D_h/2 - 1$, and $k^{(j)}$ for pair j of the key. Pair j turns at its own fixed frequency ω_j, and R_m and R_n now stand for the rotation of the whole head, pair by pair. The dot product of two head vectors is the sum of the dot products of their pairs, and identity (1) applies to each pair with $\omega = \omega_j$:

$$
\begin{aligned}
&(R_m q)^\top (R_n k) \\
&\quad = \sum_{j=0}^{D_h/2-1} \left(R(m\omega_j)\, q^{(j)}\right)^\top \left(R(n\omega_j)\, k^{(j)}\right) && \text{each pair rotated on its own} \\
&\quad = \sum_{j=0}^{D_h/2-1} q^{(j)\top} R\big((n-m)\omega_j\big)\, k^{(j)} && (1) \text{ for each pair.}
\end{aligned}
$$

Every term of the sum depends on the positions only through $n - m$, and so does the whole rotary score of the head.

What the identity covers

Identity (1) is exact for one score term while the unrotated q and k are held fixed, as A1 states (Su et al., 2024). In the network, q and k are computed from the hidden states at positions m and n. The residual states, the projections and the nonlinear layers can still carry absolute position into them. Each rotary score term is relative in this sense, but the network as a whole is not.

Only Q and K are rotated, never V. The values enter no dot product. They are the vectors that the attention weights average, so the value vectors pass unchanged into the weighted sum.

The identity also covers the position reset of packed documents. If a document's positions are not reset, every position in it is shifted by the same constant Δ. A query at $m + \Delta$ and a key at $n + \Delta$ have the offset $(n + \Delta) - (m + \Delta) = n - m$. For full-head RoPE, the rotary contribution to a score within the document depends only on that offset, so shifting one document's positions by a constant leaves those rotary score terms unchanged.

Two ways to inject position Read on your own

- Sinusoidal: added once, at the input
- **RoPE**: rotates Q and K in every layer (Su et al., 2024)

comparing the two positional strategies:

sinusoidal PE (absolute)

- added ⊕ once at the input, before block 1
- encodes absolute position
- fixed, no learnable parameters
- influences later Q/K through hidden states

RoPE (inside attention)

- rotates Q and K inside the QK product, every layer
- makes each QK score term depend on offset (n - m)
- zero trainable parameters; cosine and sine values still computed or cached
- measured overhead depends on implementation

▼ Addition and rotation are different mechanisms

The two methods differ in where position first enters the computation, and how often.

With sinusoidal encoding, the position vector is added elementwise to the token embedding once, before the first block. At position m, with token t_m, the stream entering the first block is

$$x_0 = E_{t_m} + \mathrm{PE}_m \in \mathbb{R}^D,$$

where E_{t_m} is row t_m of the embedding matrix E and PE_m is row m of Week 2's fixed sinusoidal table. The added vector encodes the absolute position m and has no learnable parameters. Later query and key projections receive positional information only through the hidden states that come from this input.

With RoPE, every attention layer first projects Q and K and reshapes them into heads. It then rotates their coordinate pairs at each token's position before the QK dot product (Su et al., 2024):

$$q_m \mapsto R_m\, q_m, \qquad k_n \mapsto R_n\, k_n \qquad \text{in every layer.}$$

Position therefore reaches every score directly, and each QK score term depends on the offset $n - m$. Under `pos_encoding: rope`, the course builds this rotation into every block. The angles are fixed, so RoPE has no trainable parameters, but its cosine and sine values are still computed or cached, and its measured overhead depends on the implementation.

Each method keeps a width unchanged, at a different place. Sinusoidal addition keeps the model width D, because E_{t_m}, PE_m and their sum all lie in $\mathbb{R}^D$. A rotation maps each head

vector in $\mathbb{R}^{D_h}$ to another vector in $\mathbb{R}^{D_h}$, so RoPE keeps the head width D_h. The widget's two panels set these insertion points side by side and list the same properties for each method.

Each pair turns at its own frequency

pair frequency

$$\omega_j = b^{-2j/d_r}$$

pair rotation

$$R(m\omega_j) = \begin{pmatrix} \cos(m\omega_j) & -\sin(m\omega_j) \\ \sin(m\omega_j) & \cos(m\omega_j) \end{pmatrix}$$

- Base `b = 10000` in the course
- Pair 0 fastest, the last pair slowest
- Fixed angles, no learned parameters

▼ Full-head pairs and their frequencies

Which coordinates form a pair

Let $b > 0$ be the fixed RoPE base and d_r the even number of rotated coordinates in one head. Week 3 uses full-head RoPE, so $d_r = D_h$. The coordinates form $d_r/2$ interleaved pairs, $(0, 1)$, $(2, 3)$ and so on up to $(d_r - 2, d_r - 1)$, indexed by $j = 0, \ldots, d_r/2 - 1$. These are the adjacent columns $(2j, 2j + 1)$ of the exercise.

Some reference code pairs the columns differently. The `rotate_half` helper in Hugging Face Transformers pairs column j with column $j + d_r/2$ (Hugging Face, 2026). Both are rotary embeddings, but weights trained under one convention give wrong scores under the other, and the exercise's tests expect adjacent pairs. Bonus 2, the course's required reading chapter on long-context positional extension, uses the same index j.

The base sets the frequencies

Definition. The first formula on the slide defines the angular frequency of pair j, in radians per position:

$$\omega_j = b^{-2j/d_r}, \qquad j = 0, \ldots, d_r/2 - 1.$$

The second formula gives the rotation of pair j at position m, by the angle $m\omega_j$.

The base and the frequencies are different quantities. The base b is one constant, and it sets how fast the frequencies fall with j. Each step from pair j to pair $j + 1$ multiplies the frequency by the same factor,

$$\frac{\omega_{j+1}}{\omega_j} = b^{-2/d_r},$$

which is less than 1 for $b > 1$. Pair 0 therefore turns fastest, at $\omega_0 = b^0 = 1$ radian per position, and the last pair, $j = d_r/2 - 1$, turns slowest, at $b^{-(d_r-2)/d_r}$. The course uses $b = 10000$, the default of `rope_theta` in its configuration.

For $d_r = 8$ and $b = 10000 = 10^4$, the four frequencies are exact powers of ten. The last column gives each pair's wavelength $2\pi/\omega_j$, the number of positions it takes to turn once:

Pair j	Columns	Exponent $-2j/d_r$	ω_j (radians per position)	Wavelength $2\pi/\omega_j$ (positions)
0	$(0, 1)$	0	$10000^0 = 1$	6.28
1	$(2, 3)$	$-1/4$	$10000^{-1/4} = 0.1$	62.8
2	$(4, 5)$	$-1/2$	$10000^{-1/2} = 0.01$	628
3	$(6, 7)$	$-3/4$	$10000^{-3/4} = 0.001$	6,283

These frequencies and the rotation matrices are fixed definitions. Nothing in them is learned, so RoPE adds no trainable parameters.

Where the rotation is applied

In the attention path, project Q, K and V first and reshape them into heads, Q into `[B,H,S,D_h]` and K and V into `[B,H_kv,S,D_h]`. Then, at token position m, apply $R(m\omega_j)$ to pair j of every query head and every key head, before the scores are computed.

Fast pairs turn quickly, slow pairs barely move

- Pair 0: one radian per position
- Shift both tokens: the angle stays

ILLUSTRATIVE position: 4 0 - 31

rotation per 2D pair, from fast (pair 0, left) to slow (pair 3, right):



pair 0: 229.2° pair 1: 22.9° pair 2: 2.3° pair 3: 0.2°

angle_j(m) = m·10000$^{-2j/d_r}$, d_r = 8 (illustrative; j = 0…3)

---

shift demo: both positions shifted by +5 (pair 0)



enclosed angle (arc): 171.9° in both cases. Angle = (n - m) × base$^{-2\cdot 0/8}$ = (7 - 4) × 1 = 3 rad, which depends only on the offset.

RoPE rotates each 2D pair by an angle proportional to the position. Pairs with a low index j rotate fast, and pairs with a high index rotate slowly. If you shift both positions by the same amount, the angle between the two vectors stays the same. For a fixed query and key, the score term of the pair shown here is therefore unchanged.

▼ What the rotation dials show

The four dials show pairs $j = 0$ to 3 of an illustrative head with $d_r = 8$ and $b = 10000$, so their frequencies are 1, 0.1, 0.01 and 0.001 radians per position. The slider sets one position m from 0 to 31 for all four dials, and dial j shows the angle $m\omega_j$. At position 0 every pair is at angle zero. Two other positions show how far apart the bands are:

Pair j	ω_j	Angle at $m = 4$	Angle at $m = 31$
0	1	4 rad $\approx 229.2°$	31 rad $\approx 1776.2°$
1	0.1	0.4 rad $\approx 22.9°$	3.1 rad $\approx 177.6°$
2	0.01	0.04 rad $\approx 2.3°$	0.31 rad $\approx 17.8°$
3	0.001	0.004 rad $\approx 0.2°$	0.031 rad $\approx 1.8°$

At position 4, pair 0 has turned more than half a turn and pair 1 about 23°, while pairs 2 and 3 have barely moved. At position 31, pair 0 has made almost five full turns, since $31/(2\pi) \approx 4.93$, and pair 3 has turned less than 2°.

The shift example below the dials uses pair 0 and the two positions $m = 4$ and $n = 7$. Shifting both by 5 moves them to 9 and 12. The offset stays $n - m = 3$, so the angle between the two rotated vectors stays $(n - m)\,\omega_0 = 3$ radians, about 171.9°, before and after the shift. The dials and vectors are synthetic coordinates chosen to show the rotation. They are neither learned Q and K values nor attention weights.

ILLUSTRATIVE position: 31 0 - 31

rotation per 2D pair, from fast (pair 0, left) to slow (pair 3, right):



pair 0: 1776.2° pair 1: 177.6° pair 2: 17.8° pair 3: 1.8°

angle_j(m) = m·10000$^{-2j/d_r}$, d_r = 8 (illustrative; j = 0…3)

shift demo: both positions shifted by +5 (pair 0)



enclosed angle (arc): 171.9° in both cases. Angle = (n - m) × base$^{-2\cdot0/8}$ = (7 - 4) × 1 = 3 rad, which depends only on the offset.

RoPE rotates each 2D pair by an angle proportional to the position. Pairs with a low index j rotate fast, and pairs with a high index rotate slowly. If you shift both positions by the same amount, the angle between the two vectors stays the same. For a fixed query and key, the score term of the pair shown here is therefore unchanged.

At position 31, pair 0 has made almost five full turns, while pair 3 has turned less than 2°.

RoPE adds no parameters but costs work Read on your own

- Parameters: zero
- Work: `O(BS(H+H_kv)d_r)` rotations
- cos and sin: computed or cached
- 1–3% overhead in one fused setup (EleutherAI, 2021)

▼ Zero parameters still require rotary work

RoPE adds no trainable parameters, because its angles $m\omega_j$ are fixed. It still costs work, in every layer.

Counting the rotations

- **A1. What is rotated.** In one attention layer, every query head and every key head is rotated at every position of every sequence. That is BSH query vectors and BSH_{kv} key vectors, held in tensors of shape `[B,H,S,D_h]` and `[B,H_kv,S,D_h]`. V is not rotated and adds nothing.
- **A2. Rotated width.** Each vector has d_r rotated coordinates in $d_r/2$ pairs. In Week 3, $d_r = D_h$.
- **A3. Cost of one pair.** Applying a 2×2 rotation to one pair takes four multiplications and two additions, once its cosine and sine are known. This counts the 2×2 product as written. A fused kernel, or one in the style of `rotate_half`, arranges the same six operations differently.

Claim (derived count). Under A1 to A3, the rotations of one layer take $3BS(H + H_{kv})\, d_r$ elementwise arithmetic operations, which is $O(BS(H + H_{kv})\, d_r)$:

$$\begin{aligned}\text{operations} &= \underbrace{BS(H + H_{kv})}_{\text{vectors}} \cdot \underbrace{d_r/2}_{\text{pairs per vector}} \cdot \underbrace{6}_{\text{per pair}} \qquad \text{A1, A2, A3} \\ &= 3\, BS(H + H_{kv})\, d_r.\end{aligned}$$

The two head counts enter separately, which matters for GQA, where H_{kv} is smaller than H and fewer key heads are rotated.

The cosine and sine tables

An implementation can compute the sine and cosine values when it needs them, read them from a table, or keep them in a cache. The angle $m\omega_j$ depends only on the position m and the pair j, so one value per position and pair is enough. That gives sine and cosine arrays of shape `[S, d_r/2]`. They broadcast across the batch and head axes, so they need not be stored once per token and head. Their actual memory footprint and traffic depend on the table length, the cache policy, the dtype, and whether the rotation is fused with neighboring operations. A

table covers only the positions it was built for, so positions must stay within the configured range. Whether a model works at positions beyond its training length is a question for evaluation, which Bonus 2 takes up.

Measured cost and quality

Reported. EleutherAI reported roughly 1–3% overhead for its fused TorchScript rotary implementation, across the transformer sizes it tested (EleutherAI, 2021). RoPE's measured cost depends on kernel fusion, dtype, hardware, tensor layout and compilation, so the range belongs to that implementation and those sizes. Other hardware and implementations need their own measurement. The quality of RoPE and how well it extrapolates to longer contexts have to be measured as well (Su et al., 2024).

29

Bonus 2 extends RoPE to longer contexts Read on your own

- Position Interpolation: rescale positions (Chen et al., 2023)
- YaRN: a different remapping and adaptation (Peng et al., 2024)
- Bonus 2: read before the Week 4 tutorial

▼ **Long contexts change the position mapping**

Two methods extend a RoPE model to contexts longer than it was trained on. Position Interpolation rescales positions (Chen et al., 2023). YaRN uses a different remapping together with its own adaptation design (Peng et al., 2024). Both change how a position m maps to the rotation angles $m\omega_j$ of its pairs. Changing that numerical mapping does not by itself teach the model to use evidence from distant tokens. Bonus 2 covers both methods. It is required reading, and the Week 4 tutorial discusses its reflection questions, so read it before then.

NoPE & the Design Space

30

No position signal at all.

Can a causal decoder tell where a token is?

Answer A6

31

Probes find position in NoPE models

- Probes recover position, studied models (Haviv et al., 2022)
- **NoPE** generalizes best: ten tasks, 100M models (Kazemnejad et al., 2023)
- Your run: one training-loss comparison

▼ What the NoPE evidence covers

Three sources speak to NoPE. Each answers its own question, for its own models.

Haviv et al. ask whether causal language models trained without an explicit positional encoding still learn where each token is. **Reported.** Probes trained on the models' representations recover absolute position in the models they study (Haviv et al., 2022). The authors offer the growing prefix, which lets a model count predecessors, as a possible explanation. The study shows that position is recoverable in those models. It identifies no circuit that every causal decoder uses, and it does not show recovery at arbitrary lengths the models have not seen.

Kazemnejad et al. ask a different question: which positional scheme generalizes best from shorter training sequences to longer evaluation sequences. **Reported.** On ten controlled algorithmic and reasoning tasks, with decoder-only models of 100M parameters, NoPE does best in their comparison (Kazemnejad et al., 2023). The result holds for those tasks, that scale and that training. It does not show that NoPE matches the other schemes in frontier models at the training length.

Your own run adds a third kind of evidence. The exercise trains a `nope` arm beside the RoPE base and measures training loss, at one sequence length, one step budget and one model scale. Report those three with the result, and keep the claim to the training-loss comparison you measured.

Hybrids mix RoPE and NoPE across layers Read on your own

- Layer-wise hybrids (Yang et al., 2025b)
- MAI-Thinking-1: five local RoPE, one global NoPE (Microsoft AI, 2026)
- One model's configuration

▼ Method evidence and one released configuration

Yang et al. study where in a model to use RoPE and where NoPE, layer by layer. **Reported.** In their evaluations, the hybrid models they train improve on long contexts and stay competitive on short ones (Yang et al., 2025b). This is evidence about the method, for the authors' models, training and evaluations. It supports studying layer-wise mixtures and gives no general recipe.

MAI-Thinking-1 is a separate, dated example of adoption. **Reported.** Its official 2026 report documents a repeating pattern of five local RoPE layers with a 512-token window, followed by one global NoPE layer (Microsoft AI, 2026). A hybrid like this applies RoPE in selected layers only, while the course's `rope` setting rotates Q and K in every layer. The pattern also makes two kinds of choice at once: the window is a locality choice, and RoPE or NoPE is a position choice. MAI-Thinking-1 shows that one released model combines them in this way. It is one model's configuration and does not make the pattern a default. When such hybrids became common, how common they are, and whether this pattern beats others would take a broader survey to establish.

Position is a configuration choice

- `rope` : the base from Week 3 on
- `sinusoidal` : the Week 2 baseline
- `nope` : ablation and design option

Position and locality are separate choices

▼ Turning the choice into a controlled comparison

The setting `pos_encoding` takes three values, and each selects its own code path:

`pos_encoding`	At the input	In every attention layer	Role in the course
`sinusoidal`	adds a position vector to the token embedding	nothing	the Week 2 baseline
`rope`	nothing	rotates Q and K after the head reshape	the base from Week 3 on
`nope`	nothing	nothing	an ablation and a design option

The sinusoidal path builds the position rows at the input, once. The RoPE path needs the rotary width and the frequency settings, and it rotates Q and K in each block. The NoPE path removes both operations and puts no new positional operator in their place.

Treat the three values as configurations of one controlled experiment. Hold everything else fixed: the architecture and parameter budget, the data order, the sequence length, the token budget, the optimizer and learning-rate schedule, the causal or packing mask, and the locality pattern. Then only the way position enters the computation varies. Record the mode, and the policy for resetting positions in packed documents, with the run, so that the comparison can be reproduced. Then compare the training-loss curves at the same number of steps and tokens.

Keep position handling apart from locality. Position handling (sinusoidal, RoPE or NoPE) decides how position reaches the attention scores. Sinusoidal encoding does it by changing the representations at the input. Locality changes which keys a query may attend to. They are separate design choices. A model can combine any choice of one kind with any choice of the other, so an effect of locality must not be credited to the positional mechanism. Week 4 varies locality.

Summary

34

Six Takeaways

1. Heads: split, attend, concatenate, `W_O`; same `4D²`
2. Fewer KV heads: cache shrinks by `H/H_kv`
3. One composed mask; loss skips document boundaries
4. `L` pre-norm blocks, one final RMSNorm
5. RoPE: rotate Q, K; score sees offset
6. NoPE: mask gives some order; locality separate

▼ How attention reads several things at once, and knows where they are

The question for today was how attention reads several things at once, and how it knows where they are.

Several heads answer the first part. The projections split their D channels into H heads of width $D_h = D/H$. Each head attends on its own, and the head contexts are transposed back, concatenated and mapped by W_O into the model width. Each query then gets H softmax rows for the $4D^2$ projection parameters of one wide head. Sharing key and value heads among the query heads shrinks the KV cache by the factor H/H_{kv} and leaves the attention work as it was. MHA, GQA and MQA form one family, set by a number H_{kv} of KV heads that divides H. The cache saving is exact, and the effect on quality has to be measured.

Real batches need one composed mask. A query-key pair is allowed when the query is a real token, the key is a real token that does not lie in the future, and both belong to the same document. That one mask makes padded and packed batches correct. The loss skips the targets that cross a document boundary as well as the padded ones. The block repeats L times in pre-norm form, and one final RMSNorm follows the stack.

For the second part, RoPE rotates the query and key coordinates of the full head in pairs, by angles proportional to each token's position. Each rotary score term then depends on the positions only through the offset $n - m$. RoPE adds no parameters, but its rotations cost work in every query and key head. A causal decoder without any position signal still gets some order from its mask alone, because the query at position t sees $t + 1$ keys. Locality, which keys a query may read at all, is a separate choice.

Six implementation checks

The six takeaways also serve as the exercise's checklist before you submit. Six implementation checks test the code behind them, each with its answer:

1. What shape must reach W_O? Concatenate the per-head contexts to restore `[B,S,D]`. The output projection W_O takes that model-width tensor.

2. Which rule aligns grouped KV heads with the query heads? Require that H_{kv} divides H, the check `n_heads % n_kv_heads == 0`. Each KV head then serves $g = H/H_{kv}$ consecutive query heads, and query head h reads KV head $\lfloor h/g \rfloor$.
3. How does a combined mask reach SDPA? As a boolean mask, in which `True` means allowed, and the composed mask goes with `is_causal=False`. An additive mask uses 0 for allowed and $-\infty$ for blocked entries.
4. Where is the last normalization? A final RMSNorm, after the loop over the blocks.
5. When is RoPE applied in the attention path? After the head reshape, rotate the paired Q and K coordinates before the scores are computed. The values are not rotated.
6. What does `pos_encoding: nope` select? No sinusoidal addition at the input and no rotary transform of Q and K. The causal mask and the other mask rules stay.

Work on the reflection questions without hints and bring your reasoning to the tutorial. Week 4 changes which keys are visible and how attention is executed: sliding windows, sinks, MLA, and FlashAttention.

To-dos until next week

Muddiest point

Do it right now! (2 min)

Before the tutorial

- Write down one question to discuss
- Think about the reflection questions

Self-study

- Read the reading slides and notes
- Read Bonus 2 before the Week 4 tutorial

Deadlines

- Quiz 1: Friday 30 October
- Second exercise: Wednesday 4 November, 22:00

Week 4 changes which keys are visible and how attention is executed: sliding windows, sinks, MLA, and FlashAttention.

Reflection questions

1. At fixed model width, what changes when the number of attention heads increases?
2. Long-context serving is KV-cache bound, and a teammate proposes moving from four KV heads to one. Using this week's arithmetic, what exactly does that change and what does it leave unchanged? Name what the arithmetic cannot tell you, and say what you would run to bound it before shipping.
3. How should causal, padding, document-boundary, and loss masks compose?
4. A packed-document run trains for hours, then returns NaN loss on a batch whose last document is unusually short. Which two mask terms are your candidates, where would you place an assertion that separates them, and why did hours of healthy training not rule this out earlier?
5. Why does rotary position embedding encode relative offsets even though it rotates queries and keys by absolute positions?
6. You must choose `pos_encoding` for a long-context run this week. Your evidence is a controlled study over ten tasks with 100M-parameter decoders and one frontier model's reported layer configuration, and neither matches your workload. What does each source actually license you to conclude, which would you weight more heavily and why, and what would you measure on your own run to find out you were wrong?

References

37

References: Attention and architectures

- Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., & Sanghai, S. (2023). GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. Proceedings of EMNLP 2023.
- DeepSeek-AI, et al. (2024). DeepSeek-V3 Technical Report. arXiv:2412.19437.
- Elhage, N., Nanda, N., Olsson, C., et al. (2021). A Mathematical Framework for Transformer Circuits. Transformer Circuits Thread.
- Ettinger, A., et al. (2025). Olmo 3. arXiv:2512.13961.
- Google. (2025). gemma_pytorch: gemma/model.py. GitHub repository, official PyTorch implementation of Gemma.
- Grattafiori, A., Dubey, A., Jauhri, A., et al. (2024). The Llama 3 Herd of Models. arXiv preprint arXiv:2407.21783.
- Olsson, C., Elhage, N., Nanda, N., et al. (2022). In-context Learning and Induction Heads. Transformer Circuits Thread.
- OpenAI. (2024 revision of the 2019 release). gpt-2: src/model.py. GitHub repository.
- OpenAI (2025). gpt-oss-120b & gpt-oss-20b Model Card. arXiv preprint arXiv:2508.10925.
- Shazeer, N. (2019). Fast Transformer Decoding: One Write-Head is All You Need. arXiv:1911.02150.
- Shazeer, N. (2020). GLU Variants Improve Transformer. arXiv:2002.05202.
- Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems 30.
- Verma, S., & Vaidya, N. (2023). Mastering LLM Techniques: Inference Optimization. NVIDIA Technical Blog.
- Yang, A., et al. (2025a). Qwen3 Technical Report. arXiv:2505.09388.

38

References: Mask APIs and model recipe

- Hugging Face. (2026). Transformers v5.14.1: rotate_half in modeling_llama.py. Source code, released 2026-07-16.
- Karpathy, A. (2026). nanochat. GitHub repository.
- PyTorch Contributors. (2026a). FlexAttention. Official documentation, accessed 2026-07-14.
- PyTorch Contributors. (2026b). MaskedTensor overview: Safe softmax. Official documentation, accessed 2026-07-14.
- PyTorch Contributors. (2026c). MultiheadAttention. Official API documentation, accessed 2026-07-14.
- PyTorch Contributors. (2026d). scaled_dot_product_attention. Official API documentation, accessed 2026-07-14.

References: Position

- EleutherAI. (2021). Rotary Embeddings: A Relative Revolution. EleutherAI Blog.
- Haviv, A., Ram, O., Press, O., Izsak, P., & Levy, O. (2022). Transformer Language Models without Positional Encodings Still Learn Positional Information. Findings of EMNLP 2022, 1382-1390.
- Kazemnejad, A., Padhi, I., Natesan Ramamurthy, K., Das, P., & Reddy, S. (2023). The Impact of Positional Encoding on Length Generalization in Transformers. Advances in Neural Information Processing Systems 36.
- Microsoft AI. (2026). MAI-Thinking-1: Building a Hill-Climbing Machine. Technical report.
- Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., & Liu, Y. (2024). RoFormer: Enhanced Transformer with Rotary Position Embedding. Neurocomputing, 568, 127063.
- Yang, B., Venkitesh, B., Talupuru, D., et al. (2025b). RoPE to NoPE and Back Again: A New Hybrid Attention Strategy. arXiv preprint arXiv:2501.18795.

References: Long-context extension

- Chen, S., Wong, S., Chen, L., & Tian, Y. (2023). Extending Context Window of Large Language Models via Positional Interpolation. arXiv:2306.15595.
- Peng, B., Quesnelle, J., Fan, H., & Shippole, E. (2024). YaRN: Efficient Context Window Extension of Large Language Models. International Conference on Learning Representations.

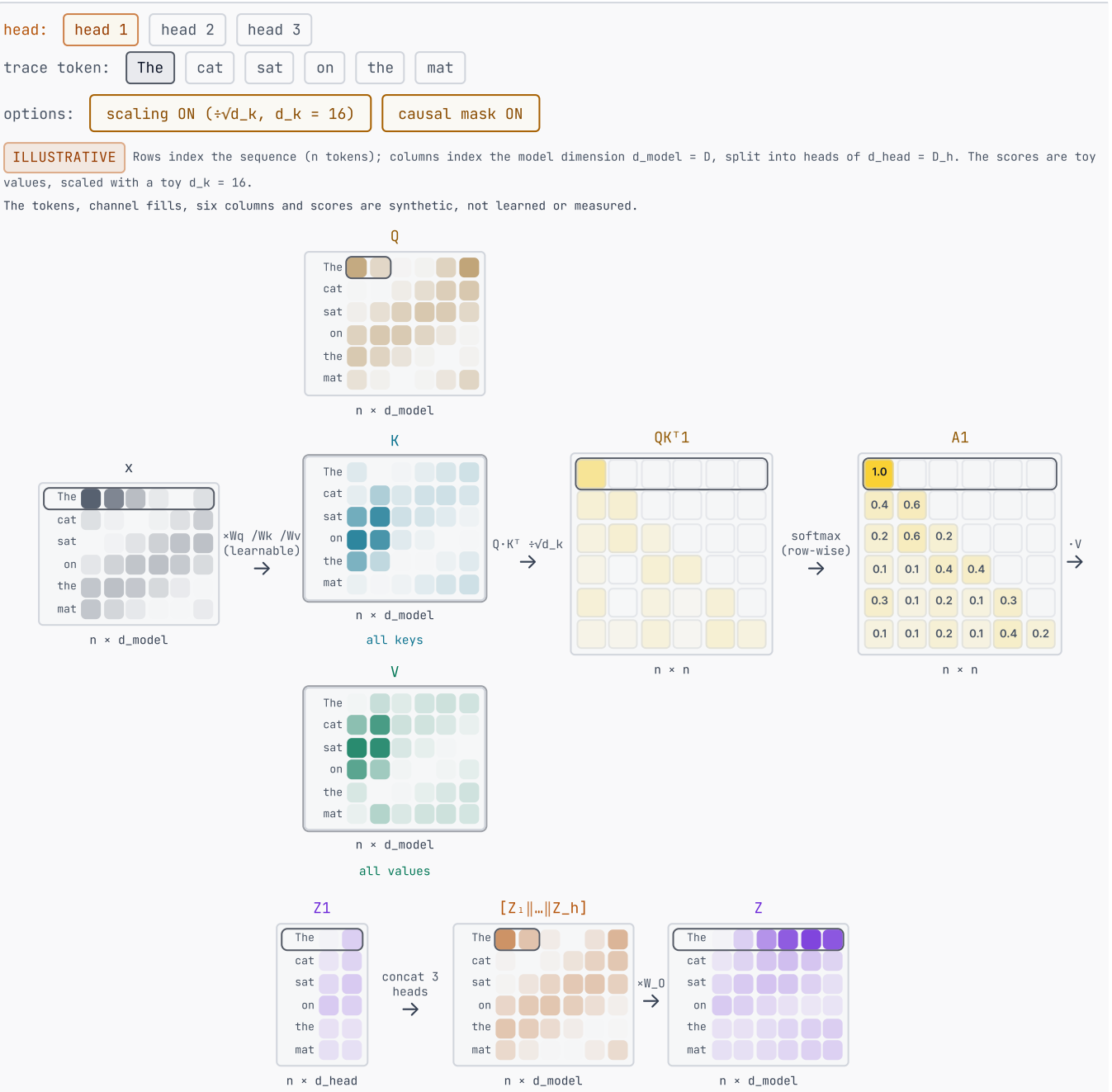
Image credits

- Kircher, A. (1679). Plan of the Labyrinth of Crete with the Minotaur at its centre. *Turris Babel*, Ichnographia labyrinthi. Internet Archive, via Wikimedia Commons. Public domain.
- Ramelli, A. (1588). Water wheel driving pumps, one shown taken apart. *Le diverse et artificiose machine*, Figure LIII. Science History Institute. Public domain.
- Ramelli, A. (1588). Fountain whose one vase feeds many spouts. *Le diverse et artificiose machine*, Figure CLXXXIIII. Science History Institute. Public domain.
- Dürer, A. (1525). Draughtsman drawing a portrait through a gridded frame. *Underweysung der Messung*, Bartsch 146. Rijksmuseum, Amsterdam. CC0.
- Ramelli, A. (1588). Gear trains on crossed shafts, with a rack and pinion. *Le diverse et artificiose machine*, Figure CLXIIII. Science History Institute. Public domain.
- Amman, J., after Jamnitzer, W. (1568). Two stellated polyhedra on turned stands. *Perspectiva corporum regularium*, F.IIII. The Metropolitan Museum of Art. CC0.
- della Bella, S. (1632). Aristotle, Ptolemy and Copernicus in conversation. *Dialogo di Galileo Galilei*, Frontispiece. The Metropolitan Museum of Art. CC0.

Answers

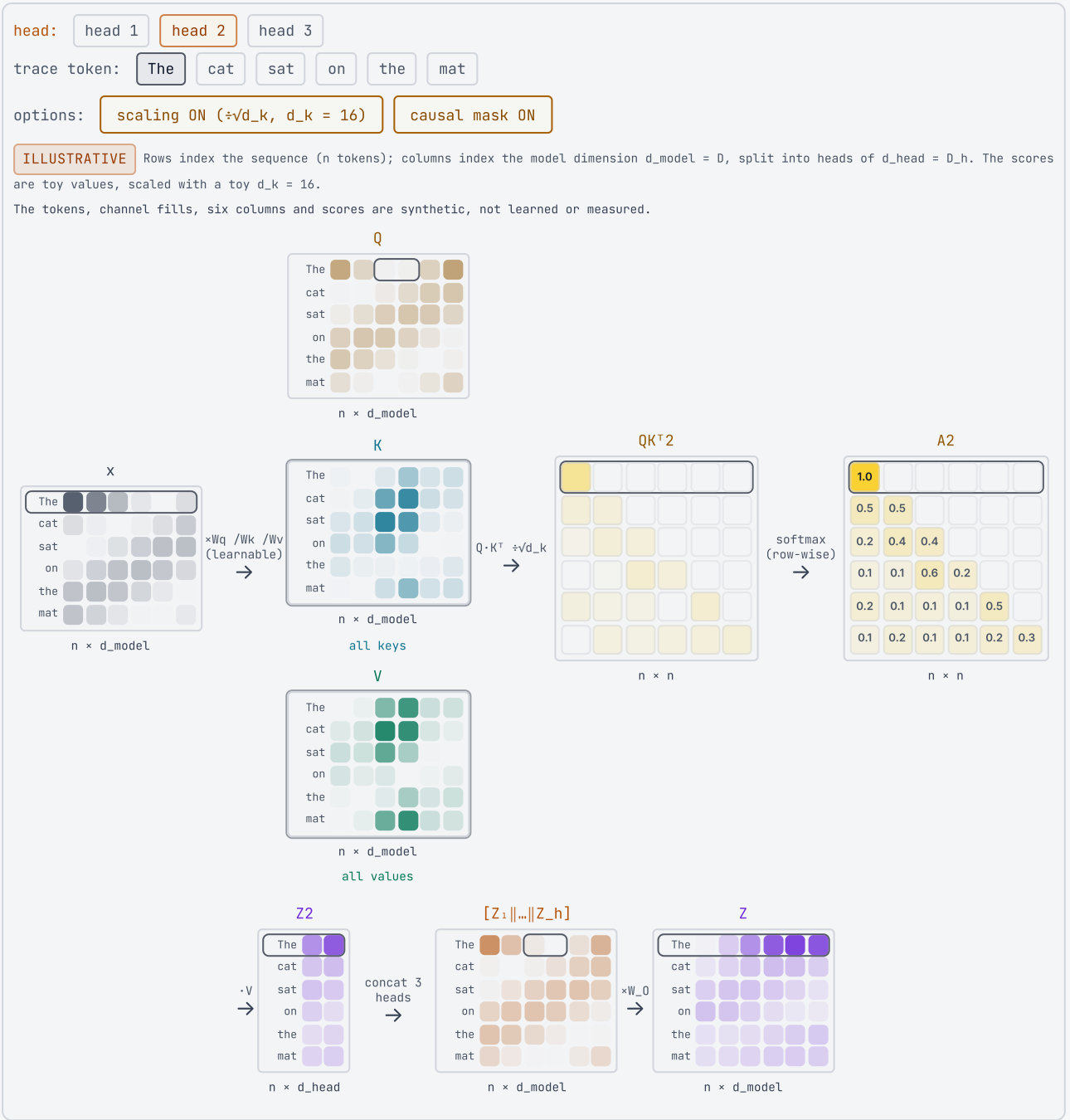
A1 · Several heads give several softmax rows

- Each head: own projections, own scores
- One softmax row per query, per head
- Concatenate the heads, then `W_O`
- No fixed job per head



Read left to right: each head computes softmax(QKᵀ ÷ √d_k) · V on its own channels. The input x is projected to Q, K and V. The scores QKᵀ go through a row-wise softmax, which gives the weights A. A then blends the values into the selected head's output. Concatenating the 3 head outputs and multiplying by W_O gives the final Z. The learned projections are D wide, and the 3 heads split them into D_h = D/H = 2 projected channels per head. In the code, the channels are reshaped from D into H × D_h and then transposed, so the head axis comes before the sequence axis. The diagram draws each head's query, key and value with D_h = 2 columns. Select a head first, then a trace token. The head control moves the highlight to the selected head's channels, now head 1's channels [0–1], and switches to that head's illustrative score pattern. These channels are a slice of the learned projections; each head sees all of x through its own weights. The token control selects the query row,

now the row of “The”. The query is compared with every key and blends every value, so all key and value rows stay available and K and V stay fully lit. QK^T, A and Z light up only the query’s own row. Toggle scaling to compare the scores before and after the division by √d_k. The scores are fixed toy values, not products of the drawn columns, so the scale uses a toy d_k = 16 whatever the drawn width. Scaling multiplies every score by 1/√d_k, so the QK^T panel dims when you turn it on. Toggle the causal mask to see the future keys (j > i) become unavailable. Change one control at a time, so that you can follow its effect along the whole path. The 3 head states are fixed illustrations, chosen to differ from one another. They d not show that real heads take on 3 fixed roles.



Head 2 on the same tokens: it reads its own slice of the channels, so its scores and attention rows differ from head 1's.

▼ Why one softmax row blends and several heads do not

A single attention head gives the query at position m one row of softmax weights a_{mn} over the keys n. Its output for that query is one weighted average of the value vectors,

$$\sum_n a_{mn}\, v_n, \qquad \text{with} \quad \sum_n a_{mn} = 1.$$

Every value feature the head gathers for this query is averaged with that one weighting. Suppose the query needs information from two positions n_1 and n_2. The head can split its weight between their two keys, say as a and $1 - a$, but its output is then

$$a\, v_{n_1} + (1 - a)\, v_{n_2},$$

one vector in which the two pieces of information arrive blended. One head can learn more than one behavior, yet for each query it produces one routing distribution.

Which ideas remove the blend

Several ideas come up when you try to remove the blend:

1. run attention several times, each time with its own query, key and value projections, so that each run computes its own scores and its own average;
2. split the model width into parts and attend within each part;
3. combine the separate results by concatenating or adding them;
4. stack a second attention layer;
5. make the softmax sharper, or keep only the two highest-scoring keys;
6. make the model wider.

The fifth and sixth ideas keep the blend. A sharper softmax or a top-two selection changes which keys receive weight, but the chosen keys are still averaged into one vector. A wider model gives that vector more channels, and each query still has one row. The fourth idea gives each query a second row, but one layer later and at the cost of depth. Within each layer, each query still has one row.

The first three ideas together are multi-head attention, with the split made in the projected channels. Each head has its own learned projections, so it computes its own score matrix, its own softmax row for each query and its own blend of the values. A query therefore gets one softmax row per head, and two heads can read two different positions without averaging them into each other. The outputs of the heads are concatenated, and an output projection W_O mixes them back into one vector for the residual stream. Several learned projection subspaces let one layer form several routing patterns and value transformations in parallel.

No head has a fixed job. Neither the architecture nor the index of a head assigns one, and training decides what each head does. Several heads give different behaviors room to coexist.

What to compare in the attention widget

The widget draws one multi-head attention layer with three heads, and the comparison it supports is between the heads. Keep the trace token fixed and switch from one head to the next. The tokens and the query position stay the same, but each head reads its own slice of the projected channels. It therefore computes its own scores and gives the same query a different softmax row. Each head's output fills its part of the concatenation, and W_O mixes all three into the layer's output `Z`. With scaling on, the widget divides the scores by $\sqrt{d_k}$ with a toy value, $d_k = 16$: the scores are fixed illustrations, not products of the drawn columns. In the model itself, each head divides by $\sqrt{D_h}$. Here D_h is the width of one head. The course sets it to the model width D divided by the number of heads H, so $D_h = D/H$.

Back to the question

A2 · Four narrow heads cost the same as one wide head

Quantity	one width-`D` head	`H` width-`D_h` heads
projection parameters	$4D^2$	$4D^2$
dense attention work	$O(S^2D)$	$O(S^2D)$
channels per head	D	D/H

- Same cost, narrower heads

▼ Counting parameters and work at fixed width

Neither layer has more parameters, and neither does more attention work. Both statements are exact counts under four assumptions, which hold everything fixed except how the width is split into heads.

- **A1. Fixed width.** The model width D is the same in both layers, because the comparison asks how to spend a given width.
- **A2. The heads fill the width.** The heads together have $HD_h = D$ channels, as in the course configuration.
- **A3. Dense, bias-free projections.** Each of the projections W_Q, W_K and W_V maps the D input channels to all HD_h head channels, and W_O maps them back to D. Every entry is a parameter, and there is no bias, as in the course model.
- **A4. The same length, dense attention.** Both layers see sequences of the same length S, and every head forms its full $S \times S$ score matrix.

Parameters

Claim. Under A1 to A3, the four projections hold $4D^2$ parameters, whatever the number of heads H.

$$\begin{aligned} P_{\text{attn}} &= \underbrace{3 \cdot D \cdot HD_h}_{W_Q, W_K, W_V} + \underbrace{HD_h \cdot D}_{W_O} && \text{A3} \\ &= 3D^2 + D^2 && \text{A2} \\ &= 4D^2. \end{aligned}$$

The head count enters only through the product HD_h, which A2 fixes at D. Each of W_Q, W_K, W_V and W_O is therefore a $D \times D$ matrix with D^2 parameters, because the heads only split the projected channels. At $D = 512$, both layers hold $4 \cdot 512^2 = 1{,}048{,}576$ attention-projection parameters.

Attention work

Count one multiply-accumulate as one MAC, which is two FLOPs. In one head and for one sequence, the scores $Q_h K_h^\top$ multiply an $S \times D_h$ matrix by a $D_h \times S$ matrix. Each of the S^2 scores is a dot product over D_h coordinates, so the product costs $S^2 D_h$ MACs. The blend multiplies the $S \times S$

weights by the $S \times D_h$ values V_h. It produces SD_h entries of S MACs each, again $S^2 D_h$. The work of one head is therefore proportional to $S^2 D_h$.

Claim. Under A2 and A4, the two attention products of one layer cost $2S^2 D$ MACs per sequence, whatever the number of heads H.

$$\begin{aligned} M_{\text{attn}} &= H \cdot 2S^2 D_h && \text{A4: } H \text{ heads, two products each} \\ &= 2S^2 \cdot H D_h \\ &= 2S^2 D && \text{A2.} \end{aligned}$$

One head of width D is the case $H = 1$ and $D_h = D$, with the same $2S^2 D$. With H heads, each of the two products costs $H \cdot S^2 D_h = S^2 D$ MACs, so both layers do $O(S^2 D)$ attention work, and a batch of B sequences multiplies the count by B. This counts matrix-multiply work and is not a model of latency. Kernels, memory traffic and how well the hardware is used can still make the two layers run at different speeds.

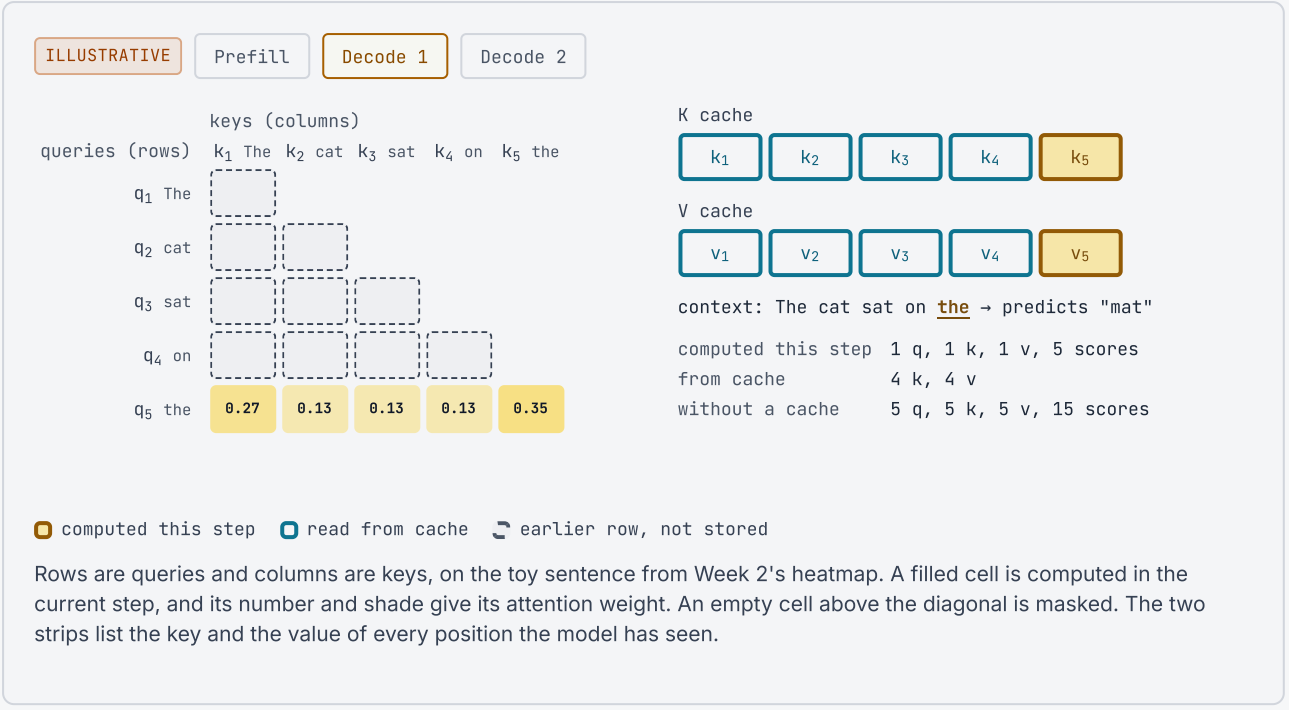
The same budget, split differently

The two layers spend the same budget in different ways. Four heads of width 128 give each query four softmax rows, each computed over 128 channels. One head of width 512 gives it one row over 512 channels. In general, H heads give each query H rows over $D_h = D/H$ channels each. Equal cost does not mean unlimited capacity or better quality: at a fixed width, every extra head makes each head narrower, and the counts say nothing about which split trains the better model.

Back to the question

A3 · Decode adds one row and reads the cache

- Prefill: the whole prompt, all rows at once
- **Decode**: one new query row per token
- **Key-value (KV) cache**: K, V of past tokens
- Earlier rows: never stored, never recomputed



Decode step 1: the query of "the" forms one new row against all five keys. Four keys and values come from the cache, and one key and one value are new.

▼ Prefill computes the prompt, decode adds one row per token

A language model generates text autoregressively: it predicts one token, appends it to the sequence and predicts again from the longer sequence. Generation starts from a prompt and runs in two phases. Prefill handles the prompt, and decode produces the new tokens one at a time. The widget shows both on the sentence of Week 2's heatmap, "The cat sat on the mat", with "The cat sat on" as the prompt.

A full model repeats Week 2's block several times, as a stack of layers. The output of one layer at a position is the input of the next layer at the same position.

Prefill runs the prompt through the model once. In every layer it computes the query, key and value of every prompt position. Its causal score matrix is the same lower triangle as in the forward pass of training: each query row holds the scores of its own key and of the keys before it. The output of the last row predicts the first new token. Prefill also writes every key and value it computed into the **key-value (KV) cache**, which keeps them for each layer and each head. In the widget, prefill computes 4 queries, 4 keys, 4 values and the 10 scores of the 4×4 triangle, and the row of "on" predicts "the".

Decode then feeds one token per step, the token just predicted. In every layer, the step computes the query, key and value of this one new position. It appends the new key and value to the cache and scores the one query against every key in the cache. That adds one new row to the score matrix. The softmax weights of this row then blend all the values in the cache. Only the new position's output goes on. It forms the new position's query, key and value in the next layer, and after the last layer it predicts the next token.

The earlier rows are never needed again. Their scores and weights formed only their own outputs. Those outputs matter to the new token only through the keys and values they form in the next layer, and those are already in that layer's cache. The next subsection shows that the causal mask keeps them valid. So the cache stores no scores, and no step recomputes an earlier row.

Why the cached keys and values are exact

The cache changes nothing in the result, and the causal mask is the reason. Number the positions from 1.

Claim. In every layer, the key and value of position i depend only on the tokens at positions 1 to i.

In the first layer, the key and value of position i are projections of its input vector, which depends only on token i and its position. Attention at position i reads only the keys and values of positions up to i, because the mask hides every later one. The other parts of the block, the normalization, the MLP and the residual addition, act on each position separately. So the output of position i depends only on the tokens at positions 1 to i, and so do the key and value it forms in the next layer. The same step, repeated layer by layer, gives the claim for every layer.

A token appended later therefore cannot change a cached key or value. The cached vectors equal what a full recomputation of the longer sequence would give. The new row's query meets the same keys and values as in that recomputation, so the new row has the same weights as the matching row of the full causal score matrix. Without the causal mask, the argument fails. From the second layer on, an earlier position would then read the appended token, so its key and value would change with every new token, and the cache would hold outdated vectors.

What the cache saves and what it costs

Decode step 1 in the widget feeds "the", the fifth token. The step computes 1 query, 1 key, 1 value and 5 scores, and it reads 4 keys and 4 values from the cache. Without a cache, each step would recompute the whole sequence: 5 queries, 5 keys, 5 values and the 15 scores of the 5×5 triangle. These counts are per head and per layer. The cache holds keys and values for every layer and every key-value head, which in MHA means every head.

In exchange, the cache grows by one key and one value per token, per layer and per key-value head, for as long as the sequence runs. Week 11 covers serving: how a server schedules prefill and decode steps across requests and manages the cache's memory.

Reading the widget

In the decode states, compare the one computed row at the bottom with the faded rows above it. Their dashed outlines and missing numbers mark the earlier rows that no step stores or recomputes. In the two strips, compare the one filled key and value, computed in this step, with the outlined ones, read

from the cache. The readout gives the counts of the step, and the context line shows the tokens so far, with the token fed in this step underlined, and names the token the computed row predicts. The decode row's weights are the same as in the matching row of the full causal matrix over all tokens so far. Week 2's heatmap, which computes the full causal matrix of the same sentence, shows the same weights in its rows for "the" and "mat". The split into prefill and decode, and the marking of each key and value as computed in the step or read from the cache, follow a figure by Verma and Vaidya (Verma & Vaidya, 2023).

Back to the question

A4 · Here the KV cache is ten times the weights

- KV cache: 512 MiB
- Weights: about 48 MiB
- Here the cache is about 10× larger

▼ Weights against cache in one serving setup

In this setup the key-value (KV) cache needs more memory than the weights: 512 MiB against about 48 MiB, roughly ten times as much (the exact ratio is $536{,}870{,}912/50{,}349{,}056 \approx 10.7$). During generation the model stores the keys and values of every past token, in every layer and for every sequence in the batch. Counting those vectors gives the size of the cache.

Definition. Take B sequences of S cached tokens and a model of L layers, each with H_{kv} key-value heads of width D_h. The **KV-cache payload** is the number of cached elements,

$$N_{\mathrm{KV}} = 2\,B\,S\,L\,H_{kv}\,D_h,$$

where the factor 2 counts keys and values. In bytes, the payload is N_{KV} times the bytes per element.

The course base model has $L = 8$ layers and $H_{kv} = 4$ key-value heads of width $D_h = 128$. The setup serves $B = 32$ sequences of $S = 1{,}024$ tokens in BF16, at 2 bytes per element, and memory is counted in MiB, with $1\,\mathrm{MiB} = 2^{20}$ bytes. The weights are the model's 25,174,528 parameters, which the Full Stack tab counts term by term, stored at 2 bytes each as well:

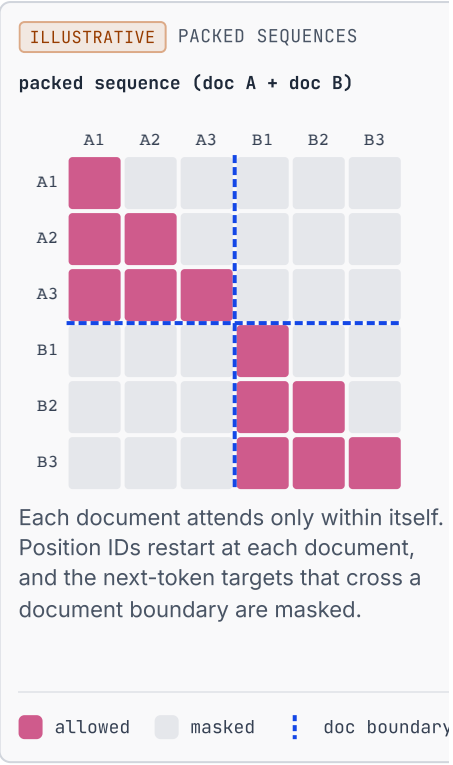
Stored tensors	Elements	Bytes	MiB
KV cache, $2 \cdot 32 \cdot 1024 \cdot 8 \cdot 4 \cdot 128$	268,435,456	536,870,912	512
Weights	25,174,528	50,349,056	≈ 48

The ratio belongs to this setup. The cache grows with the batch and the context, and the weights stay the same size. A large model serving one short sequence is dominated by its weights instead.

Back to the question

A5 · Packed documents never see each other

- B2 reads B1 and B2 only
- Block-diagonal: one block per document
- Restart positions; skip boundary targets
- Rebuild the mask for every batch



▼ Packed documents: attention, positions and the loss

B2 may read B1 and B2. Both are in its own document, and neither is in its future. The A tokens are in another document, and B3 comes after B2.

The block-diagonal mask

Applied to every query, the rule gives one lower-triangular block per document on the diagonal, and every cell outside those blocks is masked. A mask of this form is called **block-diagonal**. In the figure, rows are queries and columns are keys, and the dashed line marks the boundary between two illustrative three-token documents, A and B.

A worked row with padding

One row with padding shows all three parts together: the attention rows, the position reset and the loss. Take a six-slot row with packed IDs `[A,A,A,B,B,PAD]` and reset positions `[0,1,2,0,1,0]`. Combining a valid query, causality, a valid key and the same document gives the allowed keys below. The last two columns are the shifted next-token targets and whether the loss keeps them.

Slot	Token	Reset position	Allowed keys	Next-token target	In the loss
0	A1	0	A1	A2	yes
1	A2	1	A1, A2	A3	yes
2	A3	2	A1, A2, A3	B1	no: across the document boundary
3	B1	0	B1	B2	yes
4	B2	1	B1, B2	PAD	no: the target is padding
5	PAD	0	none	none in this row	no

Recall Week 2's loss over a set $\mathcal{T}$ of retained pairs (b, i), which averages over those pairs only:

$$\mathcal{L} = -\frac{1}{|\mathcal{T}|} \sum_{(b,i)\in\mathcal{T}} \log p_\theta(x_{b,i+1} \mid x_{b,\le i}).$$

For a packed row, $\mathcal{T}$ leaves out both the transition from A3 to B1, whose target lies across the document boundary, and the transition from B2 to PAD, whose target is padding. Tests therefore cover the attention rows, the reset positions and the loss transitions together.

Positions restart in each document

The position reset is the subtler part, because leaving it out crashes nothing. Positions restart in each document because a document's meaning does not depend on where it happened to land in the packed row. Without a reset, the first token of document B carries position 3 in this row, and position 0 in a batch that packs B first. An absolute position signal then sees the same text differently depending on its neighbors.

Whether a missing reset matters depends on the position mechanism. Sinusoidal encoding adds the position to the input vector, so a missing reset changes the input itself and every score downstream. Under full-head RoPE, the rotary terms of the scores inside a document depend only on offsets (the RoPE tab shows this), so a missing reset leaves them unchanged. NoPE uses no positions. Wherever rows are packed, every arm of a comparison must receive the same document-local positions, or the comparison is not controlled. In this week's exercise the tests check `document_position_ids` and the mask path, while the training runs use unpadded windows of one token stream. Every arm that uses positions therefore sees the default positions $0, \ldots, S-1$.

The mask changes with every batch

The document boundaries depend on what each batch packs. Carry per-batch document metadata into the mask construction, or rebuild the mask whenever the boundaries change. A fixed mask reused for a different packing layout can allow the wrong pairs without any error.

Back to the question

A6 · A causal mask alone carries some position

- Partly: query `t` sees `t + 1` keys
- Without the mask, attention is order-blind

ILLUSTRATIVE Pattern: **RoPE** NoPE

SYNTHETIC ROPE MATRIX: IDENTICAL CONTENT, COSINE SCORES

	The	cat	sat	on	the	mat
The	1.00					
cat	0.42	0.58				
sat	0.18	0.35	0.48			
on	0.10	0.16	0.31	0.43		
the	0.12	0.09	0.14	0.27	0.38	
mat	0.19	0.10	0.08	0.11	0.22	0.31

Both matrices are synthetic and give every token the same content, so only the position handling differs. Under NoPE every logit is the same, and each row spreads its weight uniformly over its causal prefix. Under RoPE each logit before the softmax is the cosine of the relative offset, so the weights rise and fall with the phase of the rotation. RoPE does not build in an ordering by recency.

ILLUSTRATIVE Pattern: RoPE NoPE

SYNTHETIC NOPE MATRIX: IDENTICAL CONTENT, UNIFORM SCORES

	The	cat	sat	on	the	mat
The	1.00					
cat	0.50	0.50				
sat	0.33	0.33	0.33			
on	0.25	0.25	0.25	0.25		
the	0.20	0.20	0.20	0.20	0.20	
mat	0.17	0.17	0.17	0.17	0.17	0.17

Both matrices are synthetic and give every token the same content, so only the position handling differs. Under NoPE every logit is the same, and each row spreads its weight uniformly over its causal prefix. Under RoPE each logit before the softmax is the cosine of the relative offset, so the weights rise and fall with the phase of the rotation. RoPE does not build in an ordering by recency.

The NoPE setting: every logit is equal, so row 0 puts all its weight on its own key and row 5 puts 1/6 on each of its six keys.

▼ What causality changes, and what it does not prove

Partly. Take a causal decoder that receives no position signal at all: no position vector at the input, and no rotation of queries and keys. The name NoPE, for no positional encoding, describes this absence. A NoPE model has no explicit positional encoding, and NoPE adds no new operator. Two facts decide what such a model can know about order. Without its mask, self-attention is blind to order. With the causal mask, different query rows see prefixes of different sizes.

Without the mask, attention is order-blind

Week 2 proved order-blindness for one head, on the slide "Without position, attention sees a set". Two of its facts carry over. Let P be an $S \times S$ permutation matrix, X the input states of one sequence, and $A(\cdot)$ the row-wise softmax. For every score matrix Z,

$$A(PZP^\top) = P\,A(Z)\,P^\top, \tag{F1}$$

and for one head without a mask or a position signal,

$$\mathrm{Attention}(PX) = P\ \mathrm{Attention}(X). \tag{F2}$$

Claim. Without a mask and without a position signal, multi-head attention is permutation-equivariant along the sequence axis: permute the input tokens, and the outputs are permuted in the same way.

Each head h computes its context C_h from its own slices of the projections, so F2 applies to every head. The heads are then concatenated along the width and multiplied by W_O from the right:

$$
\begin{aligned}
&C_h(PX) = P\,C_h(X) && \text{F2, for each head } h \\
&C(PX) \\
&\quad = \left[\, P\,C_0(X), \ldots, P\,C_{H-1}(X) \,\right] && \text{concatenation along the width} \\
&\quad = P\,C(X) \\
&C(PX)\,W_O = P\,\big(C(X)\,W_O\big) && \text{associativity.}
\end{aligned}
$$

Permuting the tokens only reorders the output rows, so the layer itself has no notion of order.

The causal mask breaks the symmetry

With the causal mask, a head's scores are $Z + M$, where M is Week 2's additive mask: $M_{tn} = 0$ for $n \le t$ and $M_{tn} = -\infty$ for $n > t$. The mask is attached to the positions, so it stays in place when the tokens are permuted, and the permuted input gives the scores $PZP^\top + M$. Week 2's step "softmax under permutation" uses F1, which needs the scores $P(Z + M)P^\top = PZP^\top + PMP^\top$. It therefore applies only if $PMP^\top = M$. The matrix $PMP^\top$ is the causal mask with its rows and columns reordered by the same permutation. It equals M only for the reordering that keeps every pair of positions in order, the identity, so for every other permutation the proof fails. Two tokens show that the property itself fails, not only the proof. In the order (a, b), the first output is v_a, because the first query sees only itself. Swap the tokens, and the first output is v_b. Equivariance would require it to equal the second output of the original order, a blend of v_a and v_b, which differs from v_b whenever $v_a \neq v_b$.

What the mask adds is a different prefix for every row. The query at position t may read its t predecessors and itself, the keys at positions $0, \ldots, t$, so it sees $t + 1$ keys. Different query rows therefore have different visible prefixes, even when every token has the same content.

Uniform weights over the prefix

The widget shows one synthetic matrix with two settings, RoPE and NoPE, and starts on RoPE. Every token has the same content vector, so both patterns come from position alone. The NoPE setting is the case of the question, and three assumptions describe it.

- **A1. No position signal.** The NoPE setting adds nothing at the input and rotates nothing, so a score depends on the content of its query and key alone.
- **A2. Identical content.** Every token has the same content vector, so every query is the same vector q, every key the same vector k and every value the same vector v. This is the widget's control: with the content removed, any difference between the rows comes from position.
- **A3. The causal mask.** The query at position t may read the keys at positions $n \le t$, and every masked entry gets the score $-\infty$.

Claim. Under A1 to A3, row t of the attention weights is uniform over its prefix:

$$a_{tn} = \begin{cases} \dfrac{1}{t+1} & n \le t, \\ 0 & n > t. \end{cases}$$

By A1 and A2, every visible score is the same number $c = q^\top k / \sqrt{D_h}$. For a visible key, $n \le t$,

$$\begin{aligned} a_{tn} &= \frac{e^{z_{tn}}}{\sum_{n'=0}^{t} e^{z_{tn'}}} && \text{softmax over the visible keys, A3} \\ &= \frac{e^{c}}{(t+1)\, e^{c}} && \text{A1 and A2: every visible score is } c \\ &= \frac{1}{t+1}, \end{aligned}$$

and a masked key, $n > t$, gets the weight $e^{-\infty} = 0$.

The constant c cancels, so the weights do not depend on the content at all. The rows differ only in the size of their prefix, and that difference comes from the mask. In the widget's six rows, row 0 puts its whole weight on its own key, and row 5 puts $1/6$ on each of its six keys.

The outputs of the head, however, are all equal in this setting. Under A2 every value is the same vector v, and a weighted average of copies of v is v itself. The prefix size reaches a head's output only once the contents differ. If the first token is a distinct start token with value v_0 and every other token has value v, the query at position t outputs

$$\frac{1}{t+1}\, v_0 + \frac{t}{t+1}\, v,$$

which changes with t.

The RoPE setting

The prefix argument above wrote the query position as t, as the slide does. The RoPE setting writes it as m, as the RoPE tab does. It uses one rotary pair, a head of width $D_h = 2$, with angular frequency $\omega = 1$. The same vector q, of unit length, serves as the query and the key at every position. Two facts give its scores. The first is the relative-score identity from the slide "Rotating q and k leaves only the offset". The second holds for any vector q in the plane: $R(\theta)q$ has the length of q and makes the angle θ with it.

$$(R_m q)^\top (R_n k) = q^\top R\big((n-m)\omega\big)\, k \tag{F3}$$

$$q^\top R(\theta)\, q = |q|^2 \cos\theta \tag{F4}$$

Claim. In the RoPE setting, the query at position m and the key at position n have the scaled logit $z_{mn} = \cos\big((n-m)\omega\big)/\sqrt{2}$. Each step uses one fact:

$$
\begin{aligned}
z_{mn} &= \frac{(R_m q)^\top (R_n q)}{\sqrt{2}} && \text{scaled score, } D_h = 2 \\
&= \frac{q^\top R\big((n-m)\omega\big)\, q}{\sqrt{2}} && \text{F3, with } k = q \\
&= \frac{|q|^2 \cos\big((n-m)\omega\big)}{\sqrt{2}} && \text{F4} \\
&= \frac{\cos\big((n-m)\omega\big)}{\sqrt{2}} && |q| = 1.
\end{aligned}
$$

For a vector whose length is not one, the numerator keeps the factor $|q|^2$.

Both settings are controls, and neither predicts the attention of a trained model. In a trained model, the queries and keys depend on the content, differ in length and direction, and combine many rotary frequencies. The learned scores then depend on the content as well as on position, and they can oscillate with the offset.

What the prefix signal can and cannot do

The different prefix sizes are a signal that training can exploit. A network could, for example, read it from the weight $1/(t+1)$ that a query puts on a distinct start token, which amounts to counting its t predecessors. Three questions remain empirical: whether a trained layer actually counts predecessors, whether the representation identifies every absolute position uniquely, and whether the signal stays reliable at lengths the model has not seen.

Back to the question